



**COMSATS University Islamabad, Sahiwal
Campus**

**SOFTWARE DESIGN
DESCRIPTION
(SDD DOCUMENT)**

for

UniLost – AI-Based University Lost & Found System

Version 1.0

By

Ahsan Ellahi CIIT/SP23-BCS-010/ISB

Supervisor

Mr. Fayez Afzaal

Bachelor of Science in Computer Science (2023-2027)

Table of Contents

1. Introduction	1
1.1 Purpose	1
1.2 Scope	1
1.3 Modules	2
2. Design methodology and software process model	4
2.1 Design Methodology	4
2.2 Software Process Model	5
3. System overview	7
3.1 System Context	7
3.2 System Modules Summary	8
3.3 System Constraints	8
4. Architectural design	9
5. Process flow/Representation	11
5.1 Major Process Flow	12
5.2 Admin Process Flow	15
6. Design models	17
6.1 Class Diagram	17
6.2 Sequence Diagram	19
6.3 State Transition Diagram	22
6.4 Data Flow Diagram	22
7. Data design	24
8. Data dictionary	27
Algorithm & Implementation	30
User Registration Algorithm	31
User Login Algorithm	31
Create Post Algorithm	32
AI Matching Algorithm	33
Ownership Verification Algorithm	33
Meetup and No-Show Detection Algorithm	34

QR Code Handover Algorithm.....	35
Chatbot Algorithm.....	36
Notification Algorithm.....	37
Search Algorithm.....	37
9. Software requirements traceability matrix	38
10. Human interface design	41
11. Screen images	41
11.1 Splash Screen	42
11.2 Welcome Screen.....	42
11.3 Login Screen	42
11.4 Signup Screen.....	43
11.5 Home Screen	43
11.6 Lost Items Screen	43
11.7 Found Items Screen.....	44
11.8 Create Post Screen	44
11.9 Item Detail Screen	44
11.10 Search Screen.....	45
11.11 Chat Screen	45
11.12 Notifications Screen	45
11.13 Profile Screen	46
11.13 My Posts Screen	46
11.14 Edit Post Screen.....	46
11.15 Edit Profile Screen	47
11.16 Settings Screen	47
12. Screen objects and actions.....	56
13. Conclusion.....	58
14. References.....	59

List of Figures

Figure 1: Agile Sprint Cycle Diagram	6
Figure 2: Unilost System Architecture.....	11
Figure 3: UniLost Student Authentication Flow	14
Figure 4: UniLost Student Main Application.....	16
Figure 5: UniLost Admin Application Flow	18
Figure 6: Unilost Class Diagram.....	20
Figure 7: Unilost Sequence Diagram	21
Figure 8: Unilost State Transition Diagram	23
Figure 9: Unilost Data Flow Diagram - Level 1	24
Figure 10: Firestore Database Diagram	26
Figure 11: Splash Screen.....	48
Figure 12: Welcome Screen	48
Figure 13: Login Screen.....	49
Figure 14: Signup Screen.....	49
Figure 15: Home Screen	50
Figure 16: Lost Item Screen.....	50
Figure 17: Found Item Screen.....	51
Figure 18: Create Post Screen.....	51
Figure 19: Item Detail Screen	52
Figure 20: Search Screen	52
Figure 21: Chat Screen.....	53
Figure 22: Notification Screen.....	53
Figure 23: Profile Screen	54
Figure 24: My Post Screen.....	54
Figure 25: Edit Post Screen.....	55
Figure 26: Edit Profile Screen.....	55
Figure 27: Settings Screen	56

List of Table

Table 1: Chats Collection	27
Table 2: Matches Collection.....	27
Table 3: Meetups Collection	28
Table 4: Messages Collection.....	28
Table 5: Notifications Collection	28
Table 6: Posts Collection.....	28
Table 7: Profiles Collection.....	29
Table 8: Reports Collection.....	29
Table 9: Trustscores Collection.....	29
Table 10: Valid_students Collection.....	30
Table 11: Verifications Collection	30
Table 12: Requirements Traceability matrix	38
Table 13: Login and Signup Screen Objects	56
Table 14: : Home Screen Objects.....	56
Table 15: Create Post Screen Objects	57
Table 16: Chat Screen Objects	57
Table 17: Notifications Screen Objects.....	57
Table 18: Search Screen Objects.....	57
Table 19: My Posts and Edit Post Objects	57
Table 20: Profile and Settings Screen Objects	58

Revision History

Name	Date	Reason for changes	Version

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken

Supervised by
Mr. Fayez Afzaal

Signature _____

1. Introduction

UniLost is an AI-powered Lost and Found system developed exclusively for COMSATS University Islamabad, Sahiwal Campus students. The system addresses the problem of lost and unclaimed items on university premises by providing a centralized, intelligent, and fully traceable digital platform.

The application serves two user roles: Students, who use a Flutter mobile application, and Administrators, who use a Flutter Web dashboard. Student access is restricted to verified university enrollees only, validated against an official student database imported from the university registrar.

The system is developed using Flutter for cross-platform mobile and web support, Firebase for authentication, real-time database, cloud functions and push notifications [1][2], Cloudinary for image storage, and Google Gemini AI for intelligent multimodal item matching and chatbot functionality [7][8] .

1.1 Purpose

The purpose of this Software Design Description (SDD) is to provide a complete technical design of the UniLost system. This document describes the architectural design, design models, data structures, algorithms, interface design, and traceability of all software requirements defined in the Software Requirements Specification (SRS).

This document serves as a reference for development, testing, and future enhancement of the UniLost system.

1.2 Scope

The UniLost system is scoped for COMSATS University Islamabad, Sahiwal Campus only. The system allows verified university students to report lost and found items, receive AI-generated match notifications, verify item ownership through a multi-layer verification process, coordinate item handover through a structured meetup and QR code system, and communicate through real-time chat.

The administrator manages the platform through a Flutter Web dashboard connected to the same Firebase backend, monitoring posts, users, matches, reports, and campus-wide analytics.

The system does not support external users, anonymous access beyond guest browsing, or any campus outside COMSATS Sahiwal.

1.3 Modules

1.3.1 Student Registration & Roll Number Verification

This module handles the registration process for new students. Students register using their university roll number which is validated against the official student database imported from the university registrar. Only valid enrolled students can create accounts.

1.3.2 User Authentication & Session Management

This module is responsible for managing secure login and user sessions. Firebase Authentication handles login using roll number and password. Session persistence ensures students remain logged in across app restarts.

1.3.3 Lost & Found Item Posting with AI-Assisted Form Filling

This module handles the creation of lost and found item posts by students. Students create posts by entering title, description, category, location, urgency level, and images. If no image is uploaded, Gemini AI asks guided smart questions to build a structured searchable description automatically [7].

1.3.4 Multimodal AI Matching Engine

This module is responsible for automatically finding potential matches between lost and found posts. When a new post is created, Firebase Cloud Functions trigger Gemini AI to compare the post against all existing posts of the opposite category using both image visual analysis and text semantic comparison. A confidence score and written match reason are generated for each potential match above 75% [5][7].

1.3.5 Multi-Layer Ownership Verification System

This module handles the verification process when a student claims a matched item is theirs. The claiming student must pass three verification layers: a knowledge challenge where Gemini generates secret questions from hidden post details, a proof submission where the student uploads an old photo or receipt, and an identity deterrence layer where the university roll number is permanently on record.

1.3.6 AI-Powered Chatbot

This module provides students with an intelligent conversational assistant powered by Gemini AI. The chatbot answers app usage questions, searches posts by text or photo, and allows students to create posts through a guided conversation without using the main form [7].

1.3.7 Real-Time Chat Between Matched Students

This module handles direct communication between two matched students. After successful verification, a private chat room is automatically created between both students where they can exchange messages, images, and coordinate the item handover.

1.3.8 Meetup Scheduling with Live GPS Tracking

This module is responsible for coordinating the physical meetup between matched students. Inside the chat, the finder proposes a meetup by selecting a campus location from a predefined list and setting a date and time. After both confirm, live GPS tracking activates for both students during the meetup window only [12].

1.3.9 QR Code Handover & Digital Receipt System

This module handles the final physical handover of the item between students. At the meetup location, the system generates a unique QR code sent to the owner only. The finder scans it to confirm handover. The system records the exact time, GPS location, and both student IDs, marks the item as returned, and sends a digital receipt to both students [13].

1.3.10 No-Show Detection & Strike System

This module is responsible for detecting and penalizing students who confirm a meetup but fail to arrive. If a confirmed meetup passes without GPS arrival detection, a grace period triggers. If the absent student still does not arrive, they receive a strike. Three strikes result in account suspension and admin notification.

1.3.11 Department-Based Urgency Notification Broadcasting

This module handles broadcasting urgent alerts when a high priority item has no match. High urgency posts with no match found within one hour trigger push notifications to all students of the same department as the poster, maximizing the chance of item recovery [6].

1.3.12 Post Reporting & Fake Post Detection

This module is responsible for maintaining the integrity and authenticity of posts on the platform. Students can report any post with a reason. Gemini AI also auto-flags suspicious posts based on vague descriptions, missing images, and new account activity. Admin reviews all reports and takes appropriate action.

1.3.13 Student Trust Score & Points System

This module handles the reputation and reward system for students on the platform. Students earn points for successfully returning items and lose trust score for strikes, false claims, and no-shows. Trust scores are visible on profiles and influence match priority.

1.3.14 Admin Dashboard

This module provides university administrators with a comprehensive web-based management interface. Administrators access a Flutter Web dashboard to manage posts, users, matches, and reports, and view campus-wide analytics including total recoveries, most common lost categories, and department-wise activity.

2. Design methodology and software process model

2.1 Design Methodology

The UniLost system is developed using the Object-Oriented Programming (OOP) methodology. OOP was selected because the system contains multiple distinct entities such as students, administrators, posts, matches, chats, meetups, verifications, and notifications, each with their own attributes, behaviors, and relationships. OOP allows these entities to be modeled as classes, making the system modular, reusable, maintainable, and scalable.

The system follows the MVVM (Model-View-ViewModel) architectural pattern combined with the Repository Pattern and GetX state management. This separates the user interface, business logic, and data handling into distinct layers, ensuring clean code organization and easier future enhancements [9].

- **Model layer:** handles data entities such as StudentModel, PostModel, ChatModel, MatchModel, MeetupModel, VerificationModel and NotificationModel [11].

- **View layer:** contains all Flutter UI screens for both student mobile app and admin web dashboard
- **ViewModel:** layer contains GetX controllers that manage state and communicate between the UI and repositories
- **Repository:** layer acts as an abstraction between Firebase services and application business logic

2.2 Software Process Model

The Agile process model is being followed for the development of UniLost [14]. Agile is an iterative and incremental software development methodology where the system is built in repeated cycles called sprints. In each sprint, a set of features is planned, designed, developed, tested, and reviewed before moving to the next sprint. This cycle continues until the complete system is delivered.

How Agile Works:

In Agile, the development process does not follow a strict linear sequence. Instead, it works in short iterative cycles. Each cycle begins with planning what features will be built, followed by designing and developing those features, testing them for correctness, reviewing the output with the supervisor, and then deploying the completed module. If changes or improvements are identified during review, they are incorporated in the next sprint. This makes Agile flexible and responsive to feedback throughout the development process.

Agile was selected for UniLost because:

- Requirements evolved during development as supervisor feedback was incorporated
- Complex features such as ownership verification, QR handover, and the strike system were refined iteratively
- Continuous testing and debugging is performed after each module is completed
- The modular nature of the system allows independent development of features in separate sprints

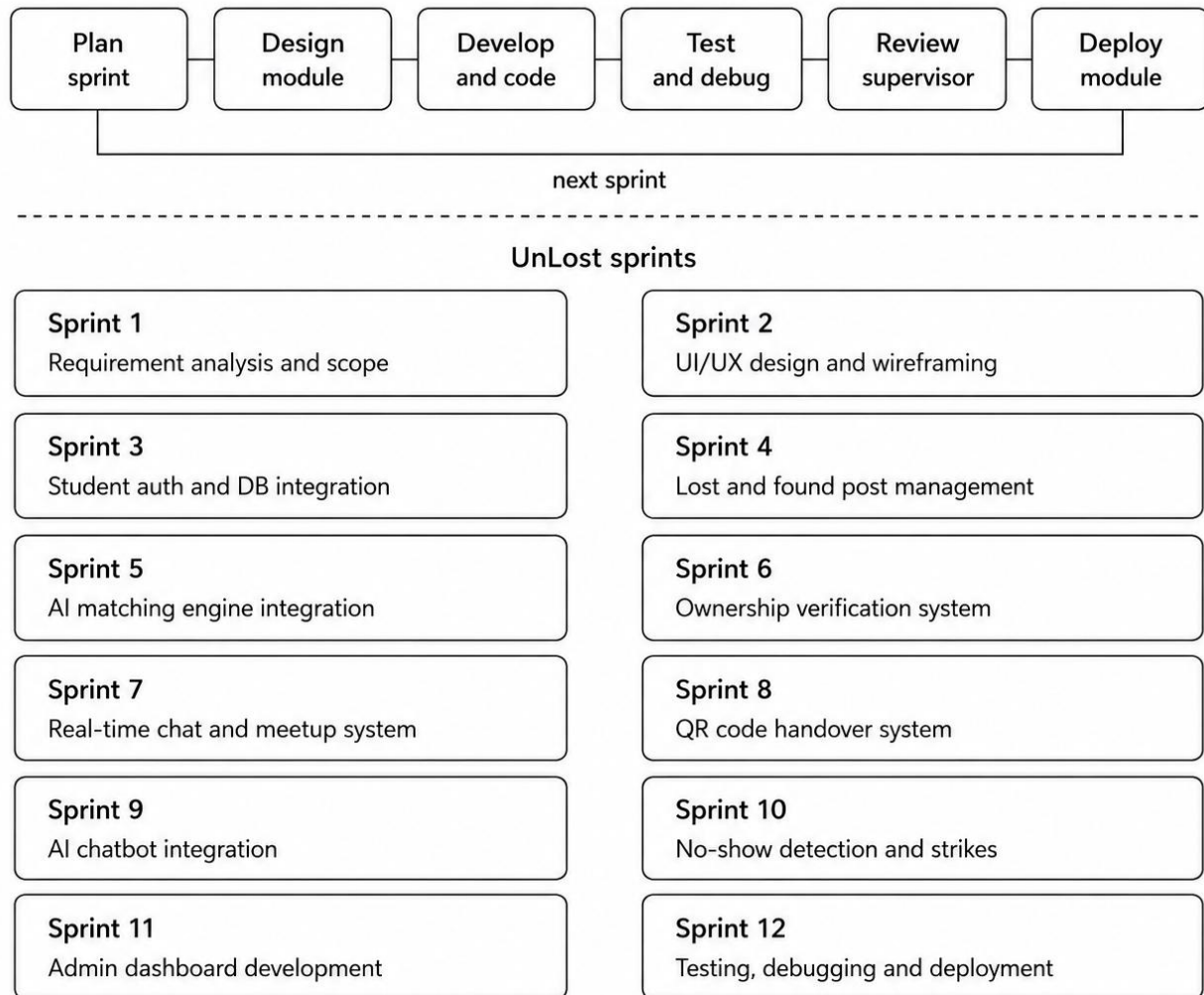


Figure 1: Agile Sprint Cycle Diagram

Figure 1 illustrates the Agile sprint cycle followed during the development of the UniLost system. The upper portion of the diagram shows the six phases of a single sprint: Plan Sprint, Design Module, Develop and Code, Test and Debug, Supervisor Review, and Deploy Module. The feedback arrow at the bottom indicates that after each sprint is deployed the team immediately begins planning the next sprint, making the process iterative and continuous. The lower portion of the diagram maps all twelve UniLost development sprints to this cycle, showing how the project was divided into incremental deliverables starting from requirement analysis and concluding with final testing, debugging, and deployment.

3. System overview

UniLost is a cross-platform AI-powered Lost and Found system designed exclusively for COMSATS University Islamabad, Sahiwal Campus. The system provides a centralized and intelligent digital platform where verified university students can report lost and found items, receive AI-generated match notifications, verify item ownership, coordinate physical handover, and communicate through real-time chat.

The system serves two distinct user roles. Students access the system through a Flutter mobile application available on Android and iOS [1]. Administrators access the system through a Flutter Web dashboard connected to the same Firebase backend, hosted on Firebase Hosting.

The core intelligence of the system is powered by Google Gemini AI, which performs multimodal matching between lost and found posts using both image visual analysis and text semantic comparison [7]. Gemini AI also powers the chatbot assistant and the ownership verification knowledge challenge.

The system is built on Firebase infrastructure including Firebase Authentication for secure login, Cloud Firestore for real-time data storage, Firebase Cloud Functions for automated backend operations, and Firebase Cloud Messaging for push notifications [2][3][4][5][6]. Cloudinary is used for cloud-based image storage and retrieval [8].

3.1 System Context

The UniLost system operates within the university environment and interacts with the following external entities:

- **University Registrar Database:** provides the official student roll number list imported into Firestore as the valid students collection
- **Google Gemini AI API:** performs item matching, chatbot responses, smart form filling, and ownership verification question generation
- **Cloudinary:** stores and serves all item images uploaded by students
- **Firebase Cloud Messaging:** delivers push notifications to student devices for matches, meetups, urgency alerts, and strikes
- **GPS Service:** provides device location during confirmed meetup windows only [12]

3.2 System Modules Summary

The UniLost system is organized into the following major functional modules:

Student Side Mobile Application

- Student registration and roll number verification
- User authentication and session management
- Lost and found item posting with AI-assisted form filling
- Multimodal AI matching engine
- Multi-layer ownership verification system
- AI-powered chatbot
- Real-time chat between matched students
- Meetup scheduling with live GPS tracking
- QR code handover and digital receipt system
- No-show detection and strike system
- Department-based urgency notification broadcasting
- Post reporting and fake post detection
- Student trust score and points system

Admin Side Flutter Web Dashboard

- Post management and moderation
- User management and strike monitoring
- Match monitoring and analytics
- Report and fake post management
- Campus-wide statistics and overview

3.3 System Constraints

- The system is accessible only to students with valid COMSATS University Islamabad Sahiwal Campus roll numbers
- Internet connectivity is required for all system operations
- Live GPS tracking is activated only during a confirmed meetup window and requires location permission from both students
- Admin access is restricted to accounts with role field set to admin in Firestore

- The system does not support anonymous posting — all posts are linked to a verified student identity

4. Architectural design

The UniLost system follows the MVVM (Model-View-ViewModel) architectural pattern combined with the Repository Pattern and GetX state management [9]. The architecture is divided into four distinct layers ensuring clean separation of concerns between the user interface, business logic, data handling, and external services.

Layer 1: View Layer (UI)

This is the topmost layer containing all Flutter UI screens visible to the user [1]. It includes all student mobile app screens such as registration, login, home feed, post creation, item detail, chat, meetup, QR handover, chatbot, notifications, profile, and settings. It also includes all admin web dashboard screens such as post management, user management, match monitoring, reports, and analytics. The View layer never directly accesses data, it only communicates with the ViewModel layer.

Layer 2: ViewModel Layer (GetX Controllers)

This layer contains all GetX controllers that manage application state and business logic. Controllers handle user interactions received from the View layer, process them, call the appropriate repository functions, and update the UI state in real time. Each major module has a dedicated controller such as AuthController, PostController, MatchController, ChatController, MeetupController, VerificationController, ChatbotController, NotificationController, and AdminController.

Layer 3: Repository Layer

This layer acts as an abstraction between the ViewModel and the Firebase services. It contains repository classes for each module that handle all data operations including reading, writing, updating, and deleting data from Firestore. The repository layer ensures that the ViewModel never directly communicates with Firebase, keeping the code clean, testable, and maintainable.

Layer 4: Data & Services Layer

This is the bottommost layer containing all external services and data sources that the system depends on:

- **Firestore Authentication:** handles student and admin login, session management, and password reset [4]
- **Cloud Firestore:** stores all application data including posts, profiles, matches, chats, meetups, verifications, notifications, and reports [3]
- **Firestore Cloud Functions:** automates backend operations such as triggering AI matching when a new post is created, no-show detection, strike issuance, and urgency notifications [5]
- **Firestore Cloud Messaging:** delivers push notifications to student devices [6]
- **Cloudinary:** stores and serves all item images uploaded by students [8]
- **Google Gemini AI API:** performs multimodal item matching, chatbot conversations, smart form filling, and ownership verification question generation [7]
- **GPS Service:** provides real-time device location during confirmed meetup windows [12]

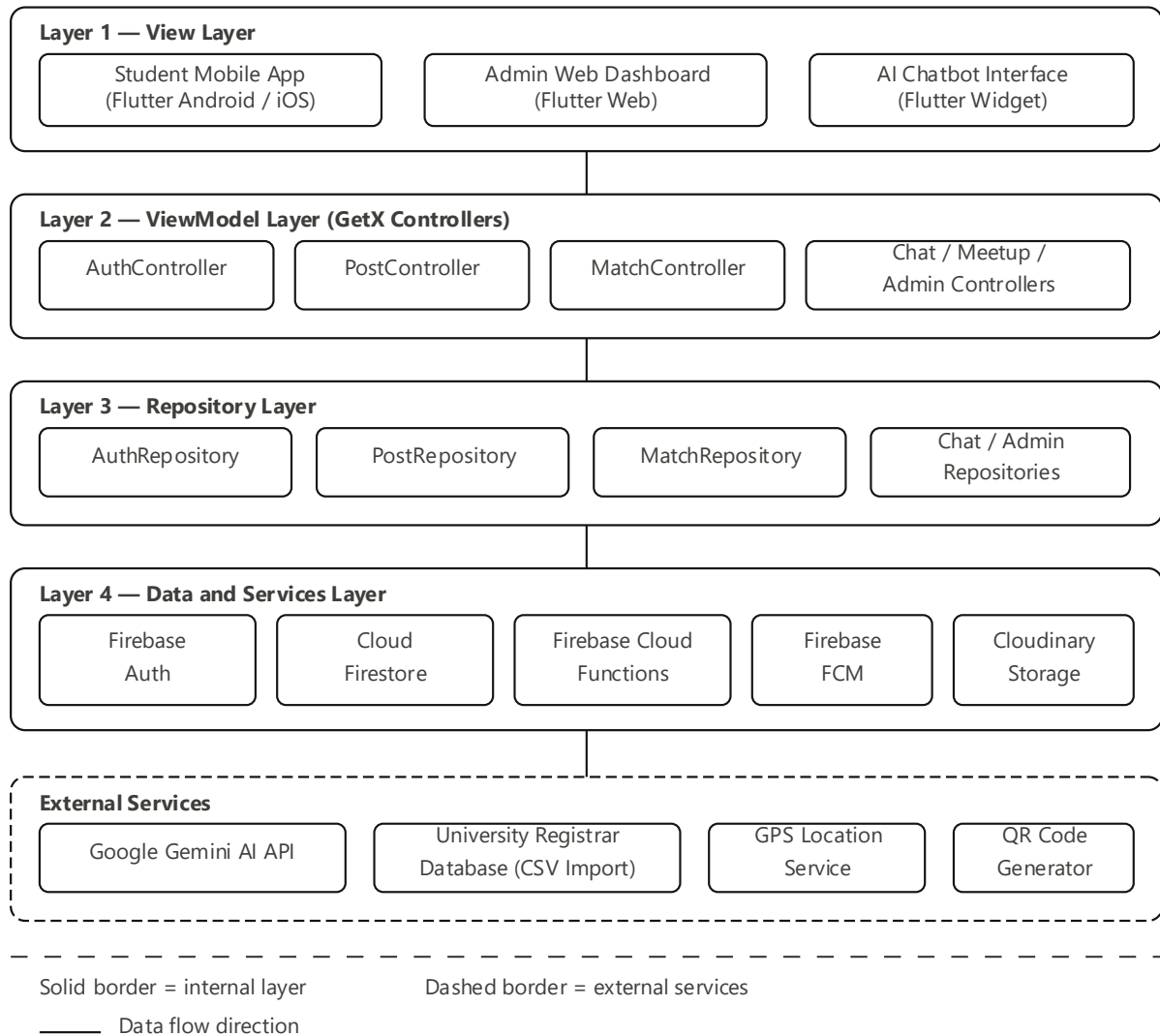


Figure 2: UniLost System Architecture

Figure 2 illustrates the layered MVVM architecture of the UniLost system. The system is divided into four layers: View, ViewModel, Repository, and Data & Services, ensuring separation of concerns between UI, business logic, data management, and external services. External services such as Google Gemini AI, GPS services, QR generation, and the University Registrar Database are integrated through the service layer to support intelligent matching and real-time system functionality.

5. Process flow/Representation

This section describes the flow of major processes in the UniLost system using activity diagrams. Two separate process flow diagrams are presented, one for the Student mobile

application and one for the Admin web dashboard. The student process flow represents the complete lifecycle of the application starting from app launch through registration, login, post creation, AI matching, ownership verification, meetup scheduling, and final QR code handover. The admin process flow represents the complete lifecycle of the admin dashboard from login through post management, user management, match monitoring, report handling, and analytics.

5.1 Major Process Flow

- **App Launch and Splash Screen :** When the application is opened the splash screen is displayed which checks the current authentication state of the user. If the user is already logged in they are redirected directly to the home screen. If not they are directed to the welcome screen.
- **Welcome Screen :** The welcome screen presents the user with two options — sign up as a new user or log in as an existing user.
- **Student Registration - Signup Flow :** New users are directed to the signup screen where they enter their university roll number. The system validates the roll number against the valid students collection in Firestore imported from the university registrar. If the roll number is invalid the registration is rejected. If valid the student fills in their full name, university email, and password. A verification email is sent to the provided email address. Once the student clicks the verification link the account is activated and the student is directed to the home screen.
- **Student Login Flow :** Existing users enter their roll number and password on the login screen. Firebase Authentication verifies the credentials [4]. If the credentials are incorrect the student is prompted to retry. If the student has forgotten their password they can request a password reset email. Upon successful authentication the student is directed to the home screen.
- **Home Screen :** After login the student lands on the home screen which displays all active lost and found posts in real time. The student can browse posts, use the search functionality, access the AI chatbot, or create a new post.
- **Post Creation :** The student creates a lost or found item post by entering the title, description, category, campus location, urgency level, and optional images. If no image is uploaded Gemini AI asks guided smart questions to build a structured description automatically. The image is uploaded to Cloudinary and post details are stored in Cloud Firestore [3][8].

- **AI Matching Engine :** When a new post is saved a Firebase Cloud Function triggers automatically sending the post to Gemini AI for multimodal comparison against all existing posts of the opposite category. A confidence score is generated for each comparison [5][7]. If no match reaches 75% the item remains active and matching continues as new posts are added.
- **Match Notification and Acceptance :** If a confidence score of 75% or above is found both students receive a push notification. Both must accept the match for the process to continue. If either student rejects the match the item remains active.
- **Ownership Verification :** The claiming student must pass the three layer ownership verification process including the knowledge challenge, proof submission, and identity deterrence. If verification fails the claim is rejected and admin is notified. If verification passes the process continues.
- **Chat and Meetup Scheduling :** A private chat room is automatically created between both students. They communicate and the finder proposes a meetup by selecting a campus location and time. Both confirm the meetup and live GPS tracking activates during the meetup window.
- **No Show Detection :** If either student fails to arrive within the grace period after the scheduled meetup time a strike is issued to the absent student, the other student is notified, and the item is re-listed as available.
- **QR Code Handover and Post Closure :** When both students arrive the system generates a unique QR code sent to the owner only. The finder scans it to confirm handover. The system records the time, GPS location, and both student IDs, marks the item as returned, sends a digital receipt to both students, awards points, and permanently closes the post.

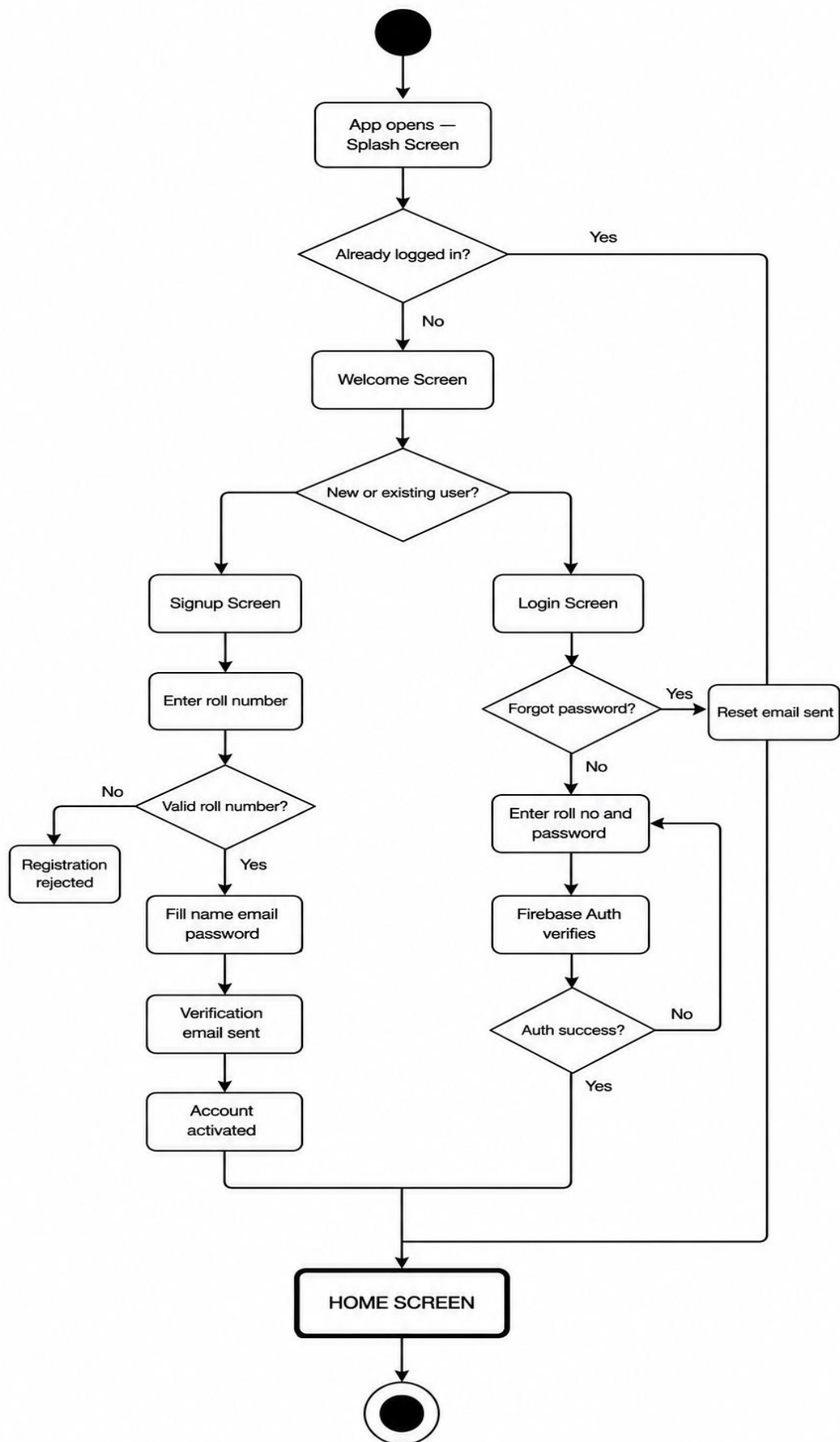


Figure 3: UniLost Student Authentication Flow

Figure 3 illustrates the complete authentication flow of the UniLost student application. The flow begins at the splash screen which checks if the user is already logged in and redirects them to the home screen if so. New users go through roll number validation, fill their details, and verify their email before their account is activated. Existing users log in using their roll number and password through Firebase Authentication with a forgot password option available.

on page 6 4: UniLost Student Main Application on page 16

5.2 Admin Process Flow

- **Admin Dashboard Launch** : The admin opens the Flutter Web dashboard and is presented with the admin login screen. The admin enters their credentials and Firebase Authentication verifies the login. The system checks the role field in Firestore and if the role is confirmed as admin the dashboard loads. If the role is student access is denied and the admin is redirected back to the login screen.
- **Dashboard Home** : After successful login the admin lands on the dashboard home screen which displays a summary of total posts, total recoveries, active reports, flagged posts, and suspended users.
- **Post Management** : The admin can view all lost and found posts in the system. Each post displays the AI risk score and student report count. The admin can view full post details, remove fake or inappropriate posts, and add notes to flagged cases.
- **User Management** : The admin can view all registered students along with their trust score, strike count, and activity history. The admin can suspend or ban accounts

Figure 5: UniLost Admin Application Flow on page 18

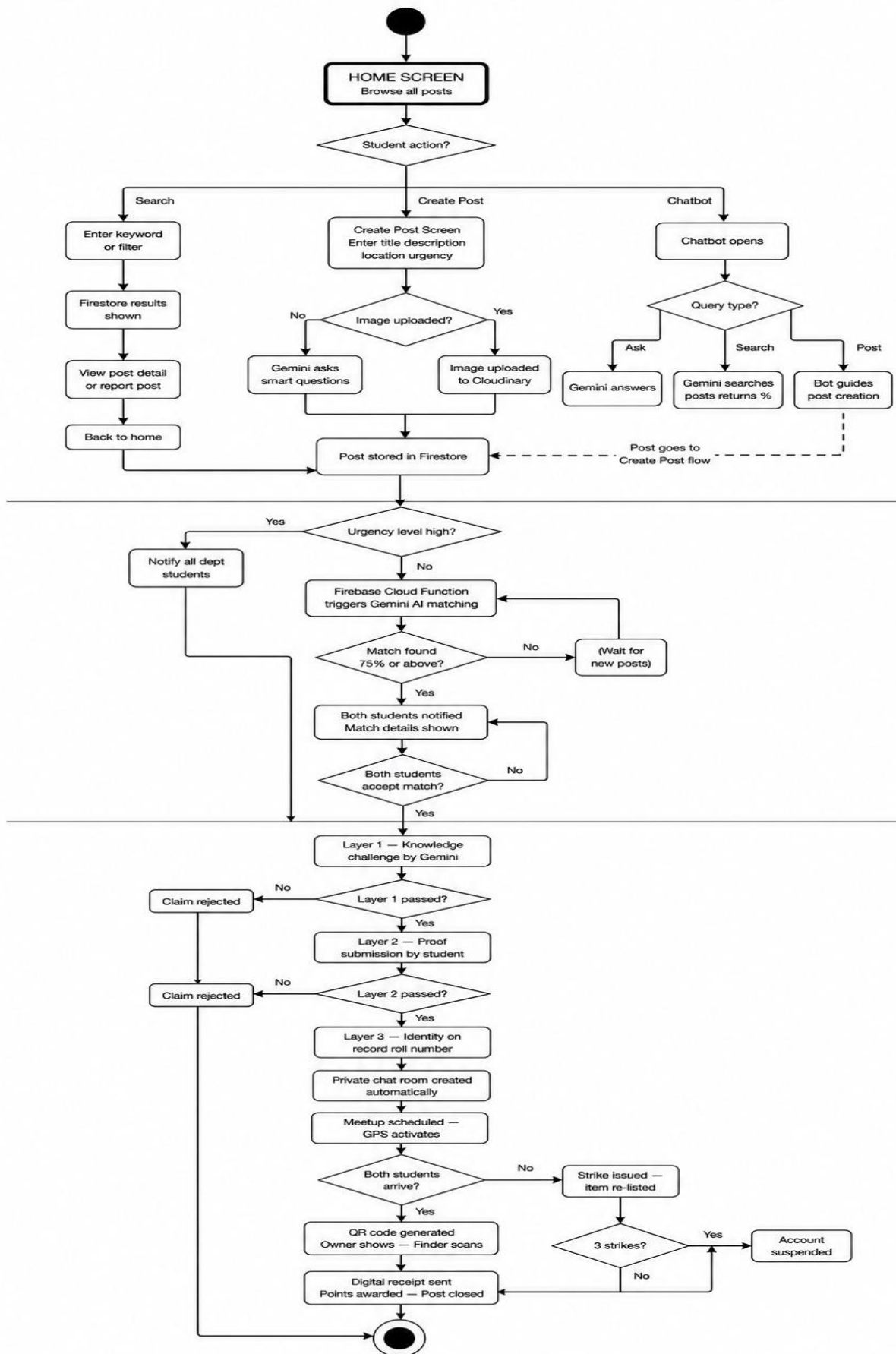


Figure 4: UniLost Student Main Application

Figure 4 illustrates the complete main application flow for the student side of UniLost starting from the home screen. The student can search posts, use the AI chatbot to ask questions or search or post items, or create a post directly. After a post is stored in Firestore the Gemini AI matching engine triggers automatically and notifies both students if a match above 75% is found. Upon mutual acceptance the student passes three ownership verification layers after which a chat room opens, a meetup is scheduled, and the item is returned through QR code handover with points awarded and the post permanently closed.

6. Design models

This section presents the design models of the UniLost system including the Class Diagram, Sequence Diagram, State Transition Diagram, and Data Flow Diagram. Each model is followed by a brief description explaining what the diagram illustrates.

6.1 Class Diagram

Description

The class diagram represents the static structure of the UniLost system showing all major classes, their attributes, methods, and relationships. The system uses object oriented principles following the MVVM architectural pattern [15].

Classes and their roles:

- **User:** parent class containing common attributes id, name, email, password, and role. Student and Admin both inherit from User through generalization.
- **Student:** inherits from User and adds rollNumber, department, batch, trustScore, strikeCount, and points. Methods include createPost(), acceptMatch(), submitVerification(), and scheduleMeetup().
- **Admin:** inherits from User. Methods include managePosts(), manageUsers(), viewAnalytics(), and handleReports().
- **Post:** contains title, description, category, location, urgencyLevel, imageUrl, status, postedByUid, and createdAt. Associated with Student through many to one association. Methods include createPost(), updatePost(), and deletePost().
- **Match:** contains matchId, lostPostId, foundPostId, lostUserUid, foundUserUid, confidenceScore, matchReason, and status. Associated with Post through aggregation.

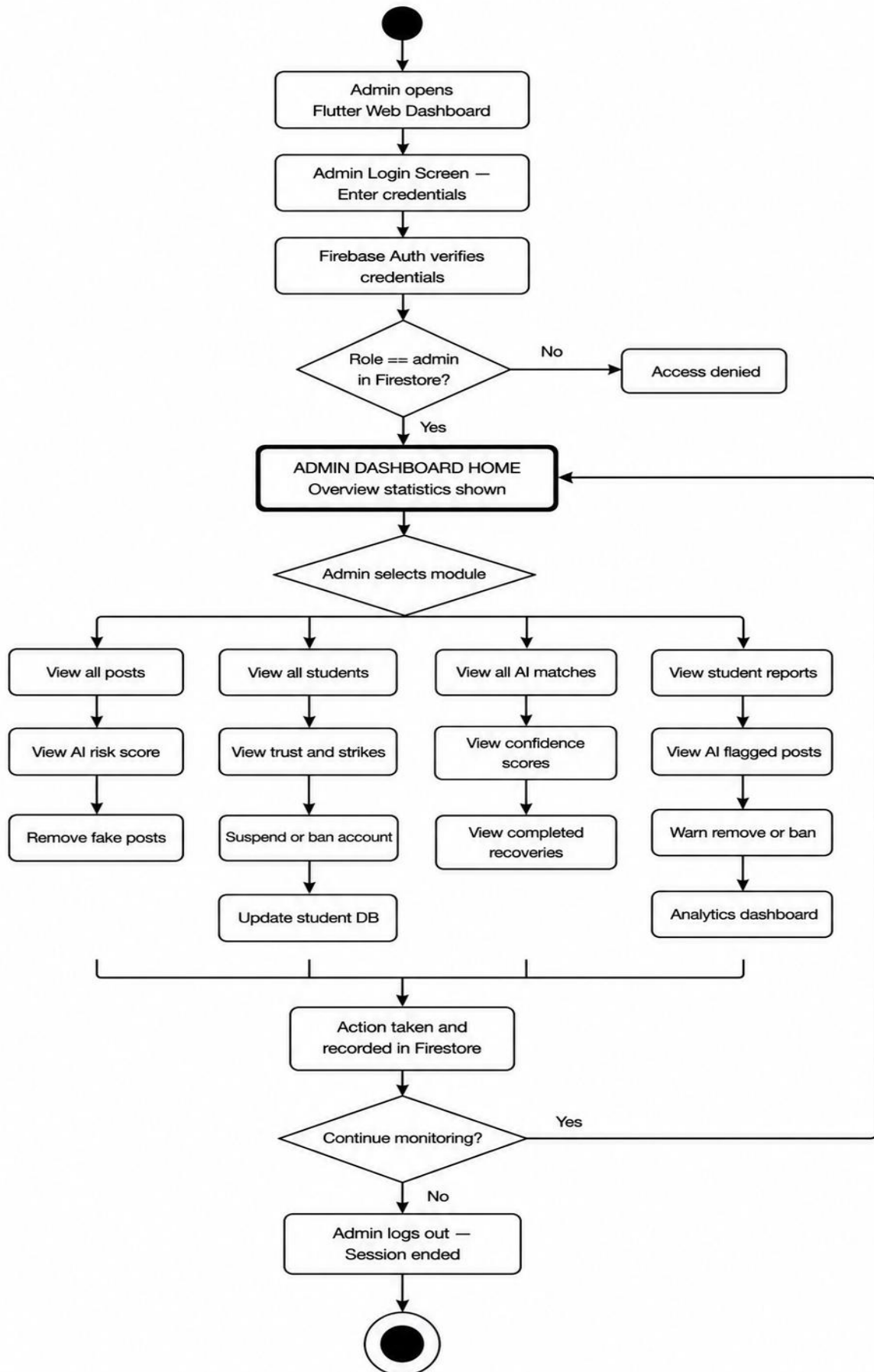


Figure 5: UniLost Admin Application Flow

Figure 5 illustrates the complete process flow for the administrator side of UniLost. After login and role verification the admin accesses the dashboard where four modules are available post management, user management, match monitoring, and reports with analytics. All actions taken are recorded in Firestore and the admin can continue monitoring or log out.

Classes and their roles:

- **Verification:** contains verificationId, matchId, claimantUid, knowledgeScore, proofImageUrl, and verificationStatus. Associated with Match through composition as verification cannot exist without a match.
- **Chat:** contains chatId, participants, itemTitle, lastMessage, lastMessageAt, matchId, and createdAt. Associated with Match through aggregation.
- **Message:** contains messageId, chatId, senderId, senderName, text, and createdAt. Associated with Chat through composition as a message cannot exist without a chat.
- **Meetup:** contains meetupId, chatId, location, scheduledTime, confirmedByOwner, confirmedByFinder, and status. Associated with Chat through aggregation.
- **Notification:** contains notificationId, recipientUid, title, body, type, matchId, isRead, and createdAt. Associated with Student through aggregation.
- **Report:** contains reportId, reportedPostId, reportedByUid, reason, and status. Associated with Post through aggregation.
- **TrustScore:** contains scoreId, studentUid, totalPoints, successfulReturns, falseClaimsCount, and noShowCount. Associated with Student through composition.

Figure 6: Unilost Class Diagram on page 20

6.2 Sequence Diagram

The sequence diagram represents the interaction between system components during the core workflow of UniLost [15], from post creation to AI matching to ownership verification to QR handover. It shows the order of messages exchanged between the Student, Flutter App, Firebase Firestore, Firebase Cloud Functions, Gemini AI, Cloudinary, and the second Student.

Figure 7: Unilost Sequence Diagram on page 21

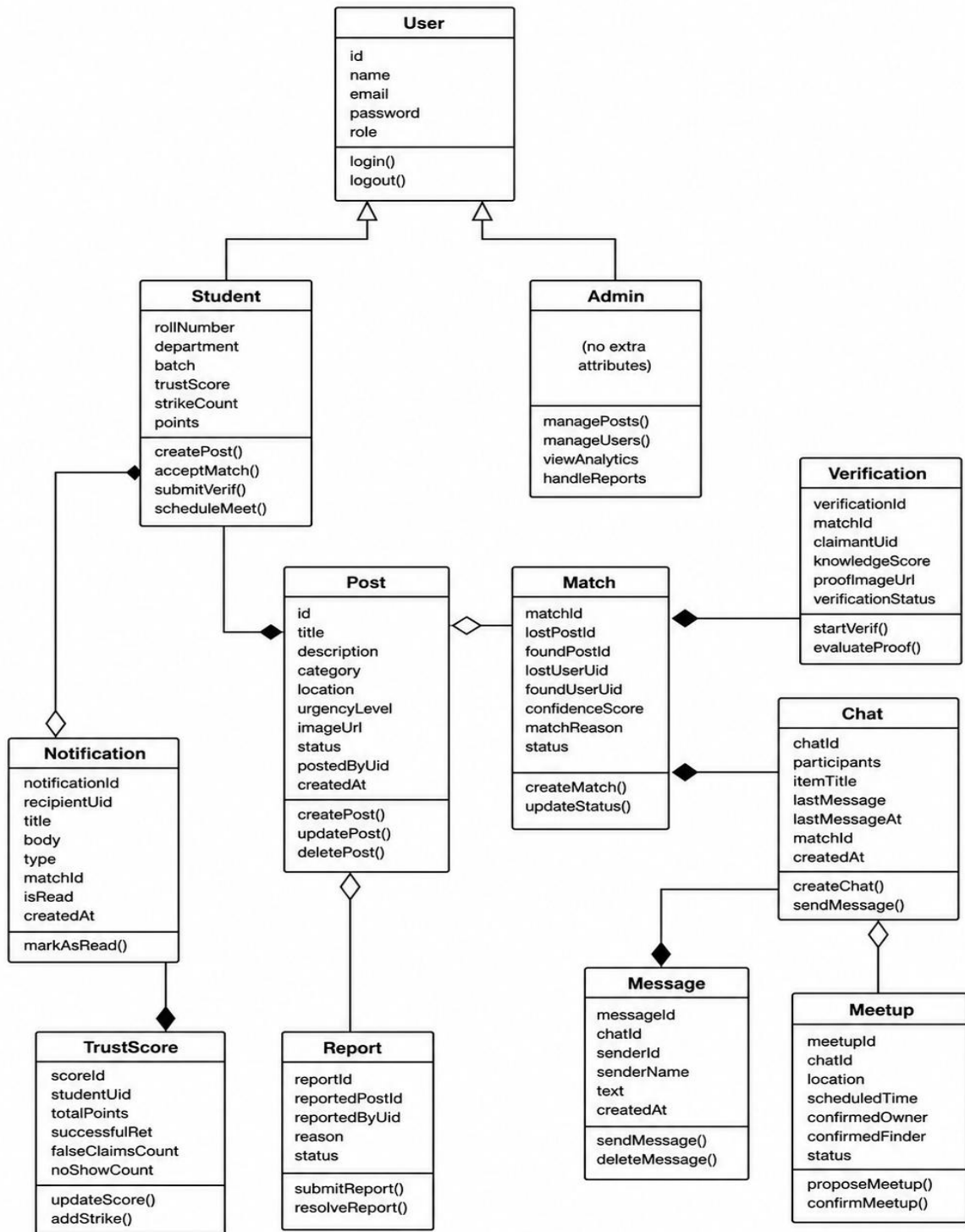


Figure 6: UniLost Class Diagram

Figure 6 illustrates the static class structure of the UniLost system. Student and Admin both inherit from the User parent class. Student is associated with Post, TrustScore, and Notification. Post is associated with Match and Report. Match uses composition with Verification and Chat. Chat uses composition with Message and aggregation with Meetup.

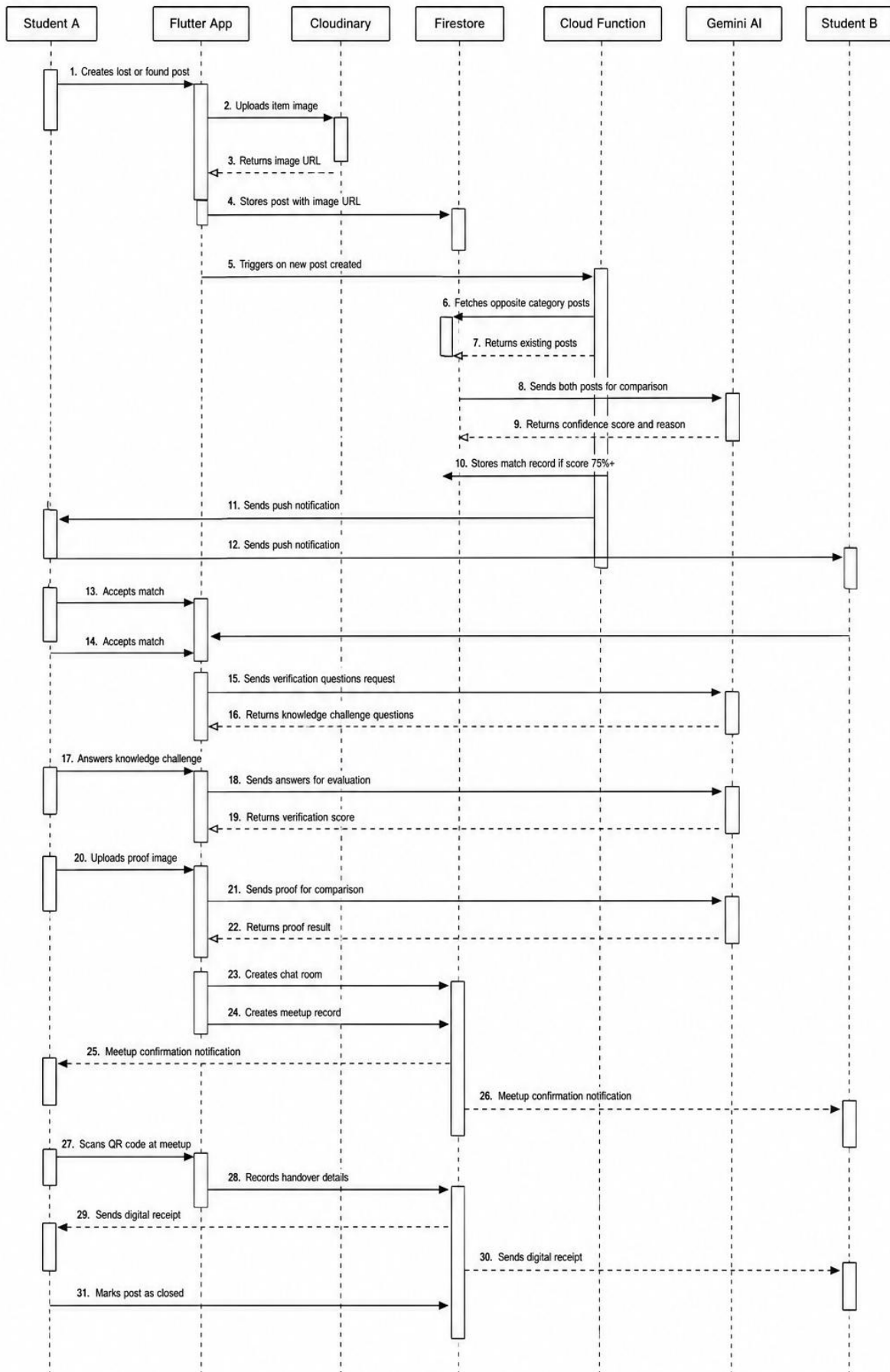


Figure 7: Unilost Sequence Diagram

Figure 7 illustrates the interaction flow between system components during the complete UniLost workflow. The sequence begins when Student A creates a post, triggering AI matching through Firebase Cloud Functions and Gemini AI. Upon a successful match both students are notified and the ownership verification process runs through Gemini AI. After verification passes a chat room and meetup are created and the workflow concludes with QR code handover and post closure.

6.3 State Transition Diagram

The state transition diagram represents the different states a lost or found item post passes through during its complete lifecycle in the UniLost system, and the events that trigger transitions between those states [15].

Figure 8: Unilost State Transition Diagram on page 23

6.4 Data Flow Diagram

The Data Flow Diagram illustrates how data moves between external entities, processes, and data stores within the UniLost system [14]. The diagram covers all major system processes including user authentication, post management, AI matching, ownership verification, chat and meetup management, QR handover, notification system, and admin management along with their interactions with ten Firestore data stores and six external entities.

Figure 9: Unilost Data Flow Diagram - Level 1 on page 24

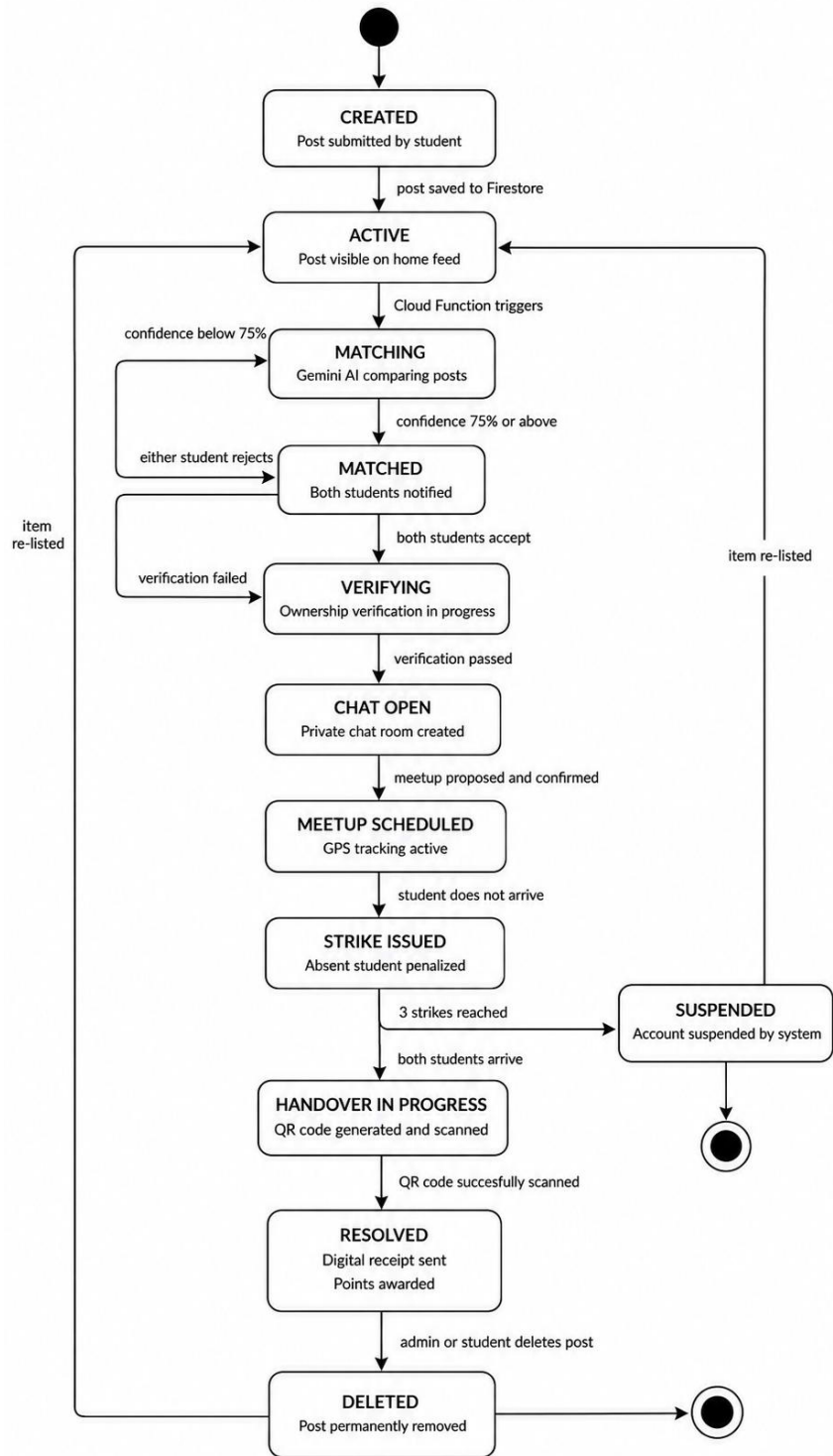


Figure 8: UniLost State Transition Diagram

Figure 8 illustrates the complete lifecycle of a lost or found item post in UniLost. A post moves from Created through Active, Matching, Matched, Verifying, Chat Open, and Meetup Scheduled states. Failed matches and rejections loop back to Active. Successful verification

and QR handover lead to Resolved. No-shows issue strikes and re-list the item while three strikes suspends the account. Posts can also be permanently deleted.

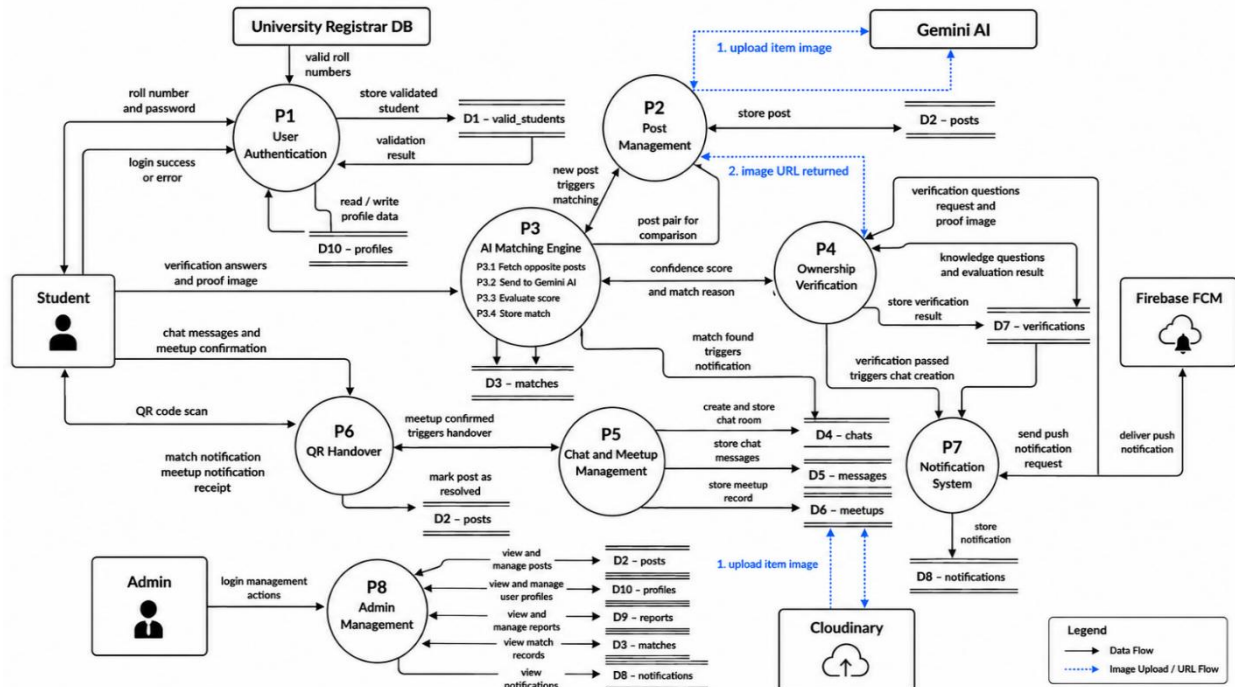


Figure 9: Unilost Data Flow Diagram - Level 1

Figure 9 illustrates the complete data flow of the UniLost system. Student and admin interactions flow through eight major processes with data stored across ten Firestore collections. Gemini AI, Cloudinary, Firebase FCM, and the University Registrar Database interact with the system at relevant process points.

7. Data design

The UniLost system uses Google Cloud Firestore as its primary database [3]. Firestore is a NoSQL real-time database that follows a collection-document architecture. Collections are automatically created when data is first written. The database is organized into ten major collections each serving a specific module of the system.

The database integrates with Firebase Authentication for user identity, Firebase Cloud Functions for automated backend operations, Cloudinary for image storage, and Google Gemini AI for intelligent matching and verification [4][5][7][8]. Firestore snapshot listeners

are used throughout the application to provide real-time updates across all connected devices without manual refresh [3].

Database Collections:

The UniLost Firestore database consists of the following ten collections:

- **Valid_students:** stores imported university student records used for registration validation
- **Profiles:** stores registered student and admin profile information
- **Posts:** stores all lost and found item posts created by students
- **Matches:** stores AI generated match records between lost and found posts
- **Verifications:** stores ownership verification records and results
- **Chats:** stores private chat room records between matched students
- **Messages:** stored as a subcollection inside each chat document containing individual messages
- **Meetups:** stores scheduled meetup records between matched students
- **Notifications:** stores system notifications for students
- **Reports:** stores student submitted reports against suspicious posts

Figure 10: Firestore Database Diagram on page 26

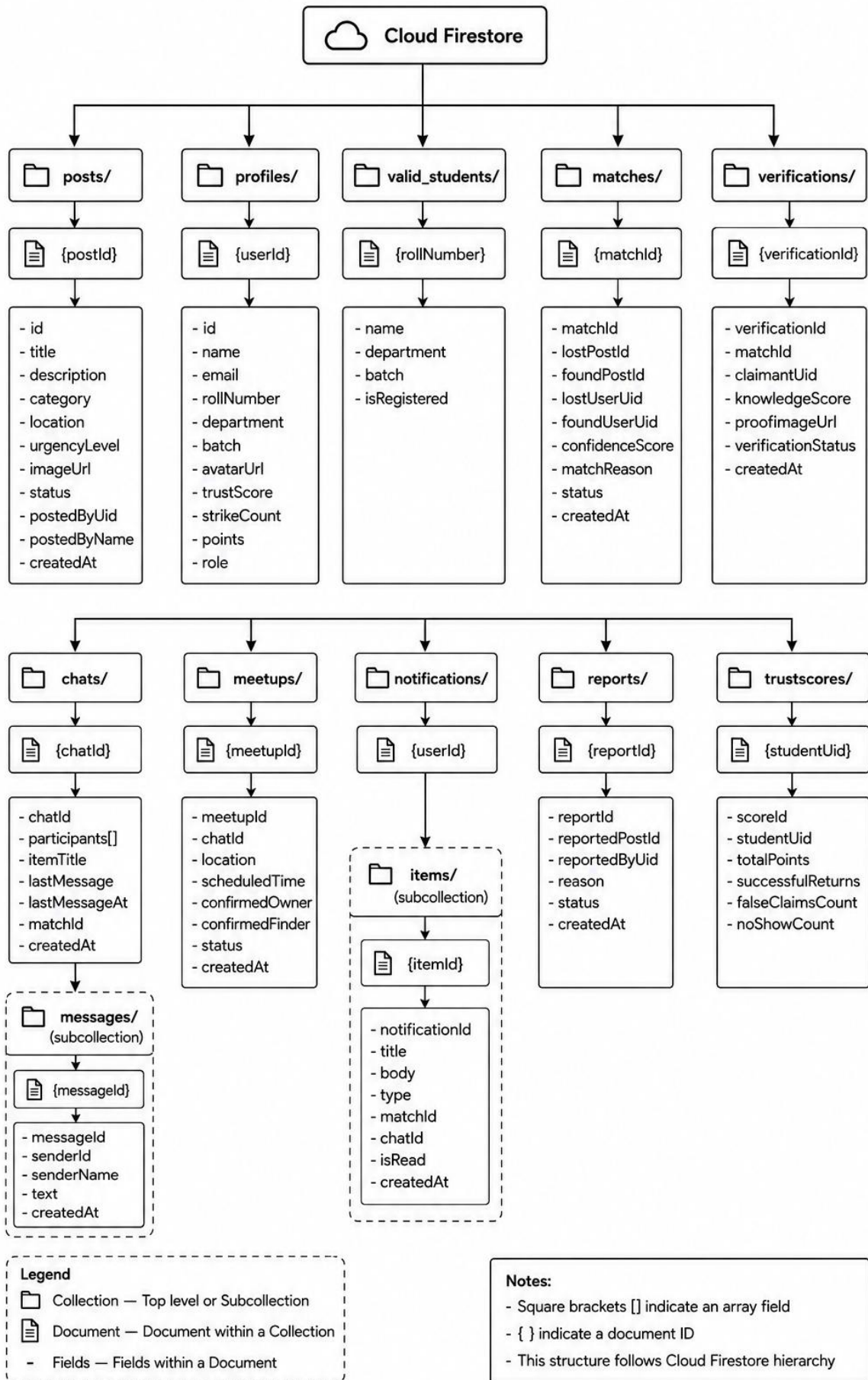


Figure 10: Firestore Database Diagram

Figure 10 illustrates the Firestore database structure of the UniLost system showing all ten collections, their documents, and fields. The messages collection exists as a subcollection inside chats and the items collection exists as a subcollection inside notifications, both shown with dashed borders.

8. Data dictionary

The data dictionary alphabetically lists the major data entities, attributes, and data types used in the UniLost system. It provides a detailed description of the data stored in Firebase Cloud Firestore collections and subcollections.

The data dictionary helps in understanding how application data is structured, stored, and processed throughout the system. Each entity includes field names, data types, and descriptions to ensure consistency and maintainability during development and future enhancements.

Table 1: Chats Collection

Attribute	Data Type	Description
chatId	String	Unique chat room ID
createdAt	Timestamp	Chat creation time
itemTitle	String	Title of matched item
lastMessage	String	Most recent message text
lastMessageAt	Timestamp	Time of last message
matchId	String	Associated match ID
Participants	Array	List of both student UIDs
chatId	String	Unique chat room ID

Table 2: Matches Collection

Attribute	Data Type	Description
confidenceScore	Double	AI generated match score
createdAt	Timestamp	Match creation time
foundPostId	String	Found item post ID
foundUserId	String	Found item owner ID
lostPostId	String	Lost item post ID
lostUserId	String	Lost item owner ID
matchId	String	Unique match ID
matchReason	String	AI generated match reason
Status	String	Current match status

Table 3: Meetups Collection

Attribute	Data Type	Description
chatId	String	Associated chat room ID
confirmedFinder	Boolean	Finder confirmation status
confirmedOwner	Boolean	Owner confirmation status
createdAt	Timestamp	Meetup creation time
Location	String	Selected campus location
meetupId	String	Unique meetup ID
scheduledTime	Timestamp	Scheduled meetup time
Status	String	Meetup current status

Table 4: Messages Collection

Attribute	Data Type	Description
createdAt	Timestamp	Message sent time
messageId	String	Unique message ID
senderId	String	Sender student UID
senderName	String	Sender full name
Text	String	Message content

Table 5: Notifications Collection

Attribute	Data Type	Description
chatId	String	Associated chat ID
createdAt	Timestamp	Notification creation time
isRead	Boolean	Read or unread status
matchId	String	Associated match ID
notificationId	String	Unique notification ID
Body	String	Notification message body
Title	String	Notification title
Type	String	Notification category

Table 6: Posts Collection

Attribute	Data Type	Description
Category	String	Lost or Found
createdAt	Timestamp	Post creation time
Description	String	Detailed item description
Id	String	Unique post ID

imageUrl	String	Clouldinary image URL
Location	String	Item lost or found location
postedByName	String	Name of post creator
postedByUid	String	UID of post creator
Status	String	Post current status
Title	String	Item title
urgencyLevel	String	High medium or low

Table 7: Profiles Collection

Attribute	Data Type	Description
avatarUrl	String	Profile image URL
Batch	String	University batch year
Department	String	Student department
Email	String	University email
Id	String	Unique user ID
Name	String	Full name
Points	Integer	Total earned points
Role	String	Student or admin
rollNumber	String	University roll number
strikeCount	Integer	Total strikes received
trustScore	Double	Student trust score

Table 8: Reports Collection

Attribute	Data Type	Description
createdAt	Timestamp	Report submission time
Reason	String	Reason for report
reportedByUid	String	Reporter student UID
reportedPostId	String	Reported post ID
reportId	String	Unique report ID
Status	String	Report current status

Table 9: Trustscores Collection

Attribute	Data Type	Description
falseClaimsCount	Integer	Number of false claims

noShowCount	Integer	Number of no shows
Scored	String	Unique score record ID
studentUid	String	Associated student UID
successfulReturns	Integer	Total successful returns
totalPoints	Integer	Total accumulated points

Table 10: Valid_students Collection

Attribute	Data Type	Description
Batch	String	University batch year
department	String	Student department
isRegistered	Boolean	Registration status
Name	String	Student full name

Table 11: Verifications Collection

Attribute	Data Type	Description
claimantUid	String	Claiming student UID
createdAt	Timestamp	Verification creation time
knowledgeScore	Double	Knowledge challenge score
matchId	String	Associated match ID
proofImageUrl	String	Submitted proof image URL
verificationId	String	Unique verification ID
verificationStatus	String	Passed or failed status

Algorithm & Implementation

This section describes the core algorithms and implementation logic of the UniLost system using pseudocode [1][2][7]. Each algorithm covers the main processing logic of a major system module.

The following pseudo code summaries describe the main processing logic of the system modules.

User Registration Algorithm

Firebase Authentication and Cloud Firestore are used to validate roll numbers and store student profiles in this algorithm. [3][4]

```
START

INPUT roll number
SEARCH valid_students collection in Firestore
IF roll number exists THEN
    SET isRegistered = false initially
    INPUT name, email, password
    SEND verification email via Firebase Auth
    IF email verified THEN
        CREATE Firebase Auth account
        STORE profile in profiles collection
        SET isRegistered = true in valid_students
        NAVIGATE to Home Screen
    ELSE
        SHOW "Please verify your email"
    ENDIF
ELSE
    SHOW "You are not a registered student"
ENDIF

END
```

User Login Algorithm

Firebase Authentication and Cloud Firestore are used to verify credentials and manage user sessions in this algorithm. [3][4]

```
START

INPUT roll number and password
FETCH email associated with roll number from
Firestore
AUTHENTICATE using Firebase Auth
IF authentication successful THEN
    CHECK role in Firestore profiles
```

```

IF role == student THEN
    NAVIGATE to Student Home Screen
ELSE IF role == admin THEN
    NAVIGATE to Admin Dashboard
ENDIF
ELSE
    SHOW login error message
    OFFER forgot password option
ENDIF

END

```

Create Post Algorithm

Cloud Firestore, Cloudinary, Google Gemini AI, and Firebase Cloud Messaging are used to store posts, upload images, generate smart questions, and send urgency notifications in this algorithm. [3][6][7][8]

```

START

INPUT title, description, category, location
INPUT urgency level
INPUT image (optional)
IF image provided THEN
    UPLOAD image to Cloudinary
    RECEIVE image URL
ELSE
    SEND smart questions to Gemini AI
    RECEIVE structured description from Gemini
ENDIF
CREATE post object with all details
STORE post in Firestore posts collection
IF urgency level == HIGH THEN
    FETCH all students of same department
    SEND push notification to all via Firebase FCM
ENDIF
TRIGGER AI matching Cloud Function
SHOW success message

END

```

AI Matching Algorithm

Firebase Cloud Functions and Google Gemini AI are used to trigger and perform multimodal matching between lost and found posts in this algorithm. [5][7]

```
START

WHEN new post created in Firestore
FETCH all active posts from opposite category
FOR each existing post DO
    SEND new post and existing post to Gemini AI
    INCLUDE title, description, location, image
    RECEIVE confidence score and match reason
    IF confidence score >= 75 THEN
        CREATE match record in Firestore
        SEND push notification to both students
        SHOW match details and confidence score
    ENDIF
ENDFOR

END
```

Ownership Verification Algorithm

Google Gemini AI and Cloud Firestore are used to generate knowledge challenge questions, evaluate proof submissions, and store verification results in this algorithm. [3][7]

```
START

BOTH students accept match
BEGIN Layer 1 — Knowledge Challenge
    FETCH found post hidden details from Firestore
    SEND details to Gemini AI
    GEMINI generates secret questions
    DISPLAY questions to claiming student
    STUDENT submits answers
    SEND answers to Gemini AI for evaluation
    RECEIVE knowledge score
    IF knowledge score >= 70 THEN
        LAYER 1 PASSED
    ELSE
```

```

    REJECT claim
    NOTIFY admin
    END
ENDIF

BEGIN Layer 2 — Proof Submission
    REQUEST proof image from student
    STUDENT uploads old photo or receipt
    SEND proof image and found post image to Gemini AI
    GEMINI compares images
    IF images consistent THEN
        LAYER 2 PASSED
    ELSE
        REJECT claim
        NOTIFY admin
        END
    ENDIF

BEGIN Layer 3 — Identity Deterrence
    RECORD student roll number permanently against
claim
    IF false claim proven later THEN
        ADMIN can take action using roll number
    ENDIF

ALL THREE LAYERS PASSED
CREATE private chat room in Firestore
NOTIFY both students

END

```

Meetup and No-Show Detection Algorithm

Cloud Firestore, Firebase Cloud Functions, and the Geolocator package are used to create meetup records, monitor GPS arrival, and issue strikes in this algorithm. [3][5][12]

```

START

FINDER proposes meetup location and time in chat
OWNER confirms meetup

```

```

CREATE meetup record in Firestore
ACTIVATE GPS tracking for both students
START countdown timer

WHEN scheduled meetup time passes
    CHECK GPS arrival status of both students
    WAIT grace period of 5 minutes
    IF both students arrived THEN
        MARK meetup as ARRIVED
        PROCEED to QR handover
    ELSE
        IDENTIFY absent student
        ISSUE strike to absent student
        INCREMENT strike count in Firestore
        NOTIFY present student
        RE-LIST item as available
        IF strike count >= 3 THEN
            SUSPEND account
            NOTIFY admin
        ENDIF
    ENDIF
ENDIF

END

```

QR Code Handover Algorithm

Cloud Firestore and the QR Code Scanner package are used to record handover details, update post status, and send digital receipts in this algorithm. [3][13]

```

START

BOTH students physically present at meetup location
SYSTEM generates unique QR code
SEND QR code to owner device only
OWNER displays QR code on screen
FINDER opens app camera
FINDER scans QR code

```

```

IF QR code valid THEN
    RECORD handover time in Firestore
    RECORD GPS location of both devices
    RECORD both student IDs
    MARK post status as RESOLVED in Firestore
    SEND digital receipt to both students
    AWARD points to both students
    UPDATE trust scores
    CLOSE post permanently
ELSE
    SHOW invalid QR code error
ENDIF

END

```

Chatbot Algorithm

Google Gemini AI and Cloud Firestore are used to process chatbot queries, search posts, and create items through guided conversation in this algorithm. [3][7]

```

START

STUDENT opens chatbot interface
INPUT query as text or image

IF query type == ASK THEN
    SEND query to Gemini AI with app knowledge context
    RECEIVE answer from Gemini
    DISPLAY answer to student

ELSE IF query type == SEARCH THEN
    FETCH all active posts from Firestore
    SEND posts data and query to Gemini AI
    GEMINI identifies matching posts
    RETURN results with match percentage
    DISPLAY results to student

ELSE IF query type == POST THEN
    GEMINI asks guided questions about item
    STUDENT provides answers
    GEMINI structures post details

```

```
PREVIEW post to student
IF student confirms THEN
    CREATE post in Firestore
    TRIGGER AI matching
ENDIF
ENDIF

END
```

Notification Algorithm

Cloud Firestore and Firebase Cloud Messaging are used to store and deliver push notifications to student devices in this algorithm. [3][6]

```
START

WHEN trigger event occurs
    match found
    match accepted
    verification result
    meetup confirmed
    no show detected
    handover complete

CREATE notification object
    SET title
    SET body
    SET type
    SET matchId or chatId
    SET isRead = false
STORE notification in Firestore notifications collection
SEND push notification request to Firebase FCM
FCM delivers push notification to student device

END
```

Search Algorithm

Cloud Firestore is used to fetch and filter posts in real time based on student search queries in this algorithm. [3]

START

INPUT search keyword or apply filter

FETCH all active posts from Firestore

FILTER posts by

title contains keyword

description contains keyword

location contains keyword

category matches filter

urgency level matches filter

DISPLAY matching results in real time

UPDATE results as new posts added

END

9. Software requirements traceability matrix

This section establishes the relationship between all functional requirements defined in the SRS document and their corresponding design components implemented in this SDD. It ensures all requirements are properly addressed in the system design.

The Software Requirements Traceability Matrix establishes a direct relationship between each functional requirement defined in the SRS document and its corresponding design component in this SDD. The matrix ensures that no requirement has been missed during the design phase and provides a clear reference for verification and testing. Each requirement is mapped to the design component that addresses it, ensuring complete coverage across all system modules including authentication, post management, AI matching, verification, chat, meetup, handover, notifications, chatbot, search, and admin management.

Table 12: Requirements Traceability matrix

Req. No	Requirement Description	Design Component	Component Item
FR01	Student registration with roll number validation	Algorithm	Registration Algorithm
FR02	User login with roll number and password	Algorithm	Login Algorithm
FR03	Forgot password and reset email	Sequence Diagram	Authentication Flow

FR04	Role based access control student and admin	Architecture Diagram	MVVM Layer Structure
FR05	Student only access via university database	Data Dictionary	valid_students Collection
FR06	Post lost item with details and image	Algorithm	Create Post Algorithm
FR07	Post found item with details and image	Algorithm	Create Post Algorithm
FR08	AI assisted form filling when noimage uploaded	Algorithm	Create Post Algorithm
FR09	Image upload to Cloundinary	Architecture Diagram	Cloundinary Integration
FR10	Multimodal AI matching text and image	Algorithm	AI Matching Algorithm
FR11	Confidence score and match reason generation	Sequence Diagram	AI Matching Workflow
FR12	Match notification to both students	Algorithm	Notification Algorithm
FR13	Match acceptance by both students	State Diagram	MATCHED State
FR14	Layer 1 knowledge challenge verification	Algorithm	Verification Algorithm
FR15	Layer 2 proof submission verification	Algorithm	Verification Algorithm
FR16	Layer 3 identity deterrence verification	Algorithm	Verification Algorithm
FR17	Automatic chat room creation after verification	Class Diagram	Chat Model
FR18	Real time messaging between students	Algorithm	Chat Algorithm
FR19	Meetup scheduling with campus location	Data Dictionary	meetups Collection
FR20	Live GPS tracking during meetup window	Architecture Diagram	GPS Service Integration
FR21	No show detection and strike issuance	Algorithm	No Show Detection Algorithm

FR22	Account suspension after three strikes	Algorithm	No Show Detection Algorithm
FR23	QR code generation and scanning	Algorithm	QR Handover Algorithm
FR24	Digital receipt sent after handover	Algorithm	QR Handover Algorithm
FR25	Post marked as resolved after handover	State Diagram	RESOLVED State
FR26	Points and trust score update after handover	Data Dictionary	trustscores Collection
FR27	AI powered chatbot for app guidance	Algorithm	Chatbot Algorithm
FR28	Chatbot search by text and photo	Algorithm	Chatbot Algorithm
FR29	Post creation through chatbot	Algorithm	Chatbot Algorithm
FR30	Search posts by keyword and filter	Algorithm	Search Algorithm
FR31	Department based urgency notification	Algorithm	Create Post Algorithm
FR32	Post reporting by student	Data Dictionary	reports Collection
FR33	AI flagging of suspicious posts	Class Diagram	Report Model
FR34	Admin post management and removal	Algorithm	Admin Flow
FR35	Admin user management and suspension	Process Flow Diagram	Admin Process Flow
FR36	Admin match monitoring	Data Dictionary	matches Collection
FR37	Admin analytics dashboard	Process Flow Diagram	Admin Process Flow
FR38	Admin reports management	Data Dictionary	reports Collection
FR39	Dark mode support	Human Interface Design	Settings Screen
FR40	Real time Firestore updates across devices	Architecture Diagram	Cloud Firestore Integration

The matrix above confirms that all forty functional requirements defined in the SRS are fully addressed within the design components of this document. Requirements spanning multiple design components are mapped to their primary component for clarity.

10. Human interface design

The UniLost application provides a clean, responsive, and user friendly interface developed using Flutter and Material Design 3 principles [1][10]. The interface is designed to ensure ease of use, accessibility, and smooth navigation across Android, iOS, and web platforms. GetX state management is used to update the interface efficiently in real time [9].

The student mobile application and admin web dashboard share the same Firebase backend but provide different interfaces tailored to their respective user roles. The student interface focuses on simplicity and quick access to core features while the admin dashboard focuses on data management and oversight.

Main Interface Features

- Responsive Flutter based cross platform user interface
- Material Design 3 implementation
- Bottom navigation bar for major student modules
- Real time Firestore updates without manual refresh
- Dark mode support via settings toggle
- Outlined text fields for visual consistency
- Custom snackbar notifications for user feedback
- AI assisted interaction flow throughout the app
- Role based interface separation for student and admin

11. Screen images

The following screens represent the currently implemented modules of the UniLost system. These are not the final screens and may change as additional modules are completed. Remaining screens including match confirmation, ownership verification, meetup scheduling, QR handover, chatbot interface, and admin dashboard are under development and will be included in the final version of this document.

11.1 Splash Screen

Image on page 48on page 48

Description

The splash screen is displayed when the application starts. It shows the animated COMSATS logo and checks the current authentication state of the user. Logged-in users are redirected to the home screen, while new users are redirected to the welcome screen.

Features

- Animated logo transition
- Authentication state checking
- Automatic navigation
- Smooth startup experience

11.2 Welcome Screen

Image on page 48on page 48

Description

The welcome screen introduces users to the UniLost application and provides navigation options for login, signup, or guest mode access.

Features

- Application introduction
- Login navigation
- Signup navigation
- Guest mode access

11.3 Login Screen

Image on page 49on page 49

Description

The login screen allows users to authenticate using their university roll number and password. Firebase Authentication is used for secure login and session persistence.

Features

- Roll number based login
- Password authentication
- Forgot password option
- Guest mode support

- Input validation

11.4 Signup Screen

Image on page 49

Description

The signup screen allows new users to create an account by entering personal and university-related information. User data is stored in Firebase Authentication and Firestore.

Features

- User registration
- Roll number registration
- Email and phone input
- Password validation
- Firestore profile creation

11.5 Home Screen

Image on page 50

Description

The home screen displays the real-time feed of lost and found posts retrieved from Cloud Firestore. Users can browse posts, search items, and navigate to other modules.

Features

- Real-time posts feed
- Lost and found item cards
- Search functionality
- Shared AppBar navigation
- Bottom navigation bar

11.6 Lost Items Screen

Image on page 50on page 50

Description

The lost items screen displays all posts categorized as lost items. Data is retrieved in real time from Firestore using category-based filtering.

Features

- Lost items filtering
- Real-time Firestore updates

- Item browsing
- Navigation to item details

11.7 Found Items Screen

Image on page 51

Description

The found items screen displays all found item posts available in the system. Users can browse found items and contact the uploader.

Features

- Found items filtering
- Real-time updates
- Item browsing
- Contact functionality

11.8 Create Post Screen

Image on page 51on page 51

Description

The create post screen allows users to create lost or found item posts by entering item details and uploading an image. Images are uploaded to Cloudinary before storing the post in Firestore.

Features

- Item detail input
- Image upload
- Cloudinary integration
- Firestore post creation
- AI matching trigger

11.9 Item Detail Screen

Image on page 52on page 52

Description

The item detail screen displays complete information about a selected item including image, location, and contact options.

Features

- Large image preview
- Detailed item information

- Contact functionality
- Real-time data display

11.10 Search Screen

Image on page 52

Description

The search screen allows users to search lost and found posts using keywords related to title, description, or location.

Features

- Real-time searching
- Keyword filtering
- Firestore integration
- Dynamic search results

11.11 Chat Screen

Image on page 53

Description

The chat screen provides real-time messaging functionality between users using Firestore streams. Messages are updated instantly without requiring manual refresh.

Features

- Real-time messaging
- Firestore synchronization
- AI-generated system messages
- Instant message updates
- Conversation history

11.12 Notifications Screen

Image on page 53

Description

The notifications screen displays AI-generated alerts and system notifications related to item matches and activities.

Features

- Match notifications
- Read/unread status

- Real-time updates
- Notification management

11.13 Profile Screen

Image on page 54on page 54

Description

The profile screen displays user profile information, activity statistics, and navigation options for settings and account management.

Features

- User information display
- Match statistics
- Posts statistics
- Edit profile option
- Logout functionality

11.13 My Posts Screen

Image on page 54

Description

The My Posts screen displays all posts created by the currently logged-in user.

Features

- User-specific posts
- Edit post option
- Delete post option
- Firestore synchronization

11.14 Edit Post Screen

Image on page 55on page 55

Description

The edit post screen allows users to modify previously created lost or found item posts.

Features

- Update item details
- Replace item image
- Firestore update
- Data validation

11.15 Edit Profile Screen

Image on page 55

Description

The edit profile screen allows users to update their personal information stored in Firestore.

Features

- Name editing
- Phone number update
- Roll number update
- Real-time Firestore update

11.16 Settings Screen

Image on page 56

Description

The settings screen allows users to customize application preferences including dark mode, notifications, and AI matching settings.

Features

- Dark mode toggle
- Notification settings
- AI suggestions toggle
- Application preferences
- Settings persistence

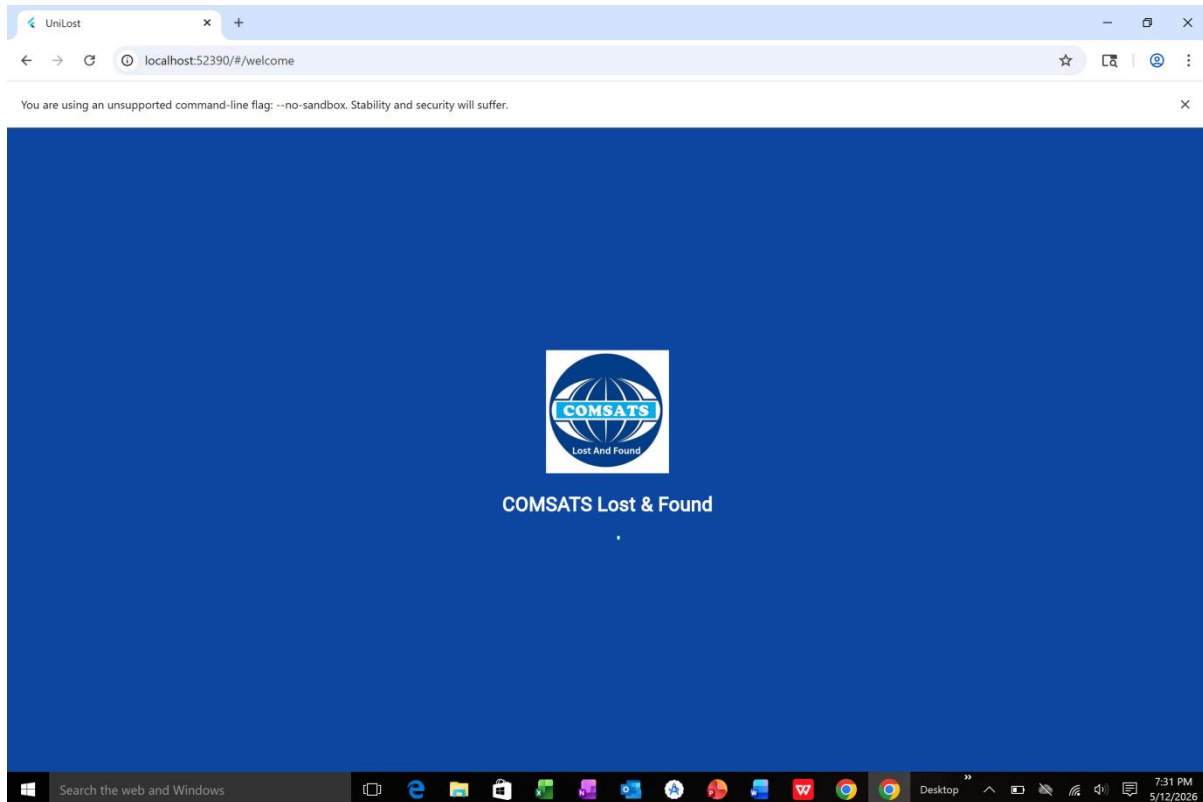


Figure 11: Splash Screen

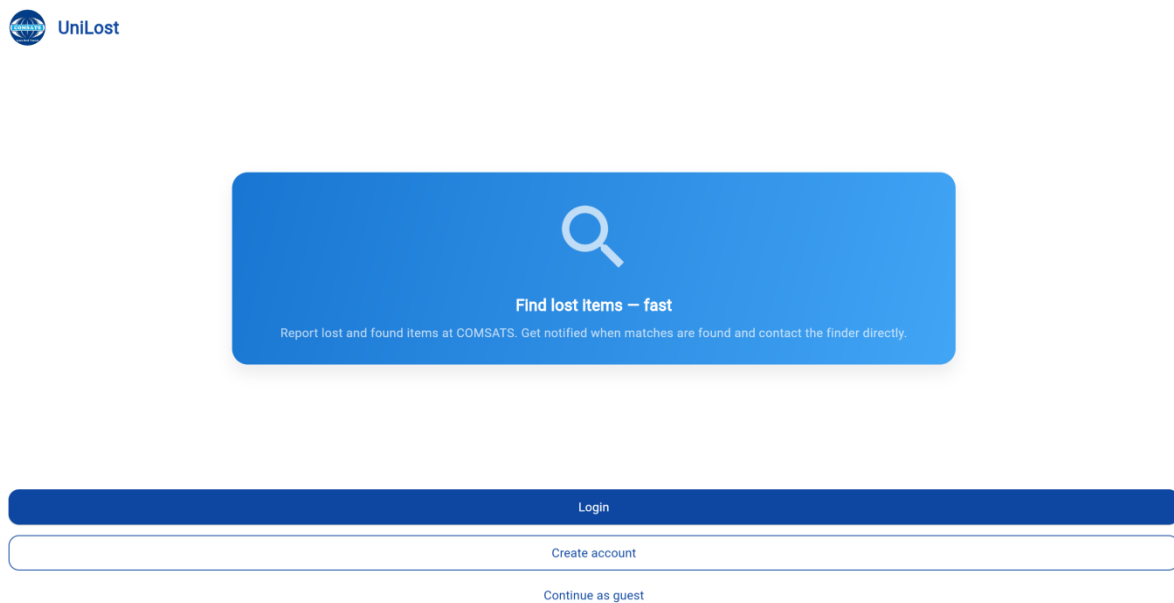


Figure 12: Welcome Screen

[←](#) Login



COMSATS
Unit And Front

Welcome to UniLost
Login with your roll number

 Roll Number

 Password 

Login

[Don't have an account? Sign up](#)
[Forgot password? Reset via email](#)

Figure 13: Login Screen

[←](#) Create Account



COMSATS
Unit And Front

 Full Name

 Email

 Phone Number

 Roll Number

 Password 

Sign Up

[Already have an account? Login](#)

Figure 14: Signup Screen



Figure 15: Home Screen



Figure 16: Lost Item Screen

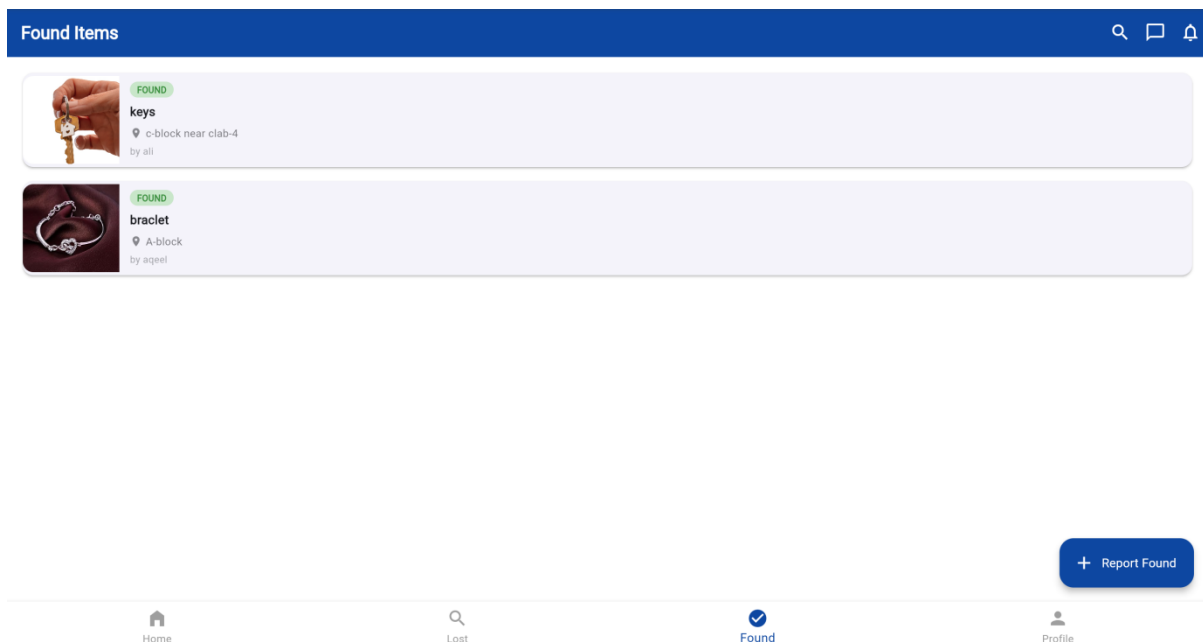


Figure 17: Found Item Screen

The screenshot displays the 'Report Lost Item' screen. At the top, a blue header bar contains a back arrow and the text 'Report Lost Item'. Below the header, there are two buttons: 'Lost' (with a checkmark) and 'Found'. The 'Lost' button is selected. Below these buttons, there is a section for 'Item Photo'. It includes a placeholder image with a plus sign and the text 'Tap to add'. To the right of the placeholder, there is a text input field for the photo. Below the photo section, there are three text input fields: 'Title', 'Description', and 'Location'. At the bottom, there is a large blue button with the text 'Submit'.

Figure 18: Create Post Screen

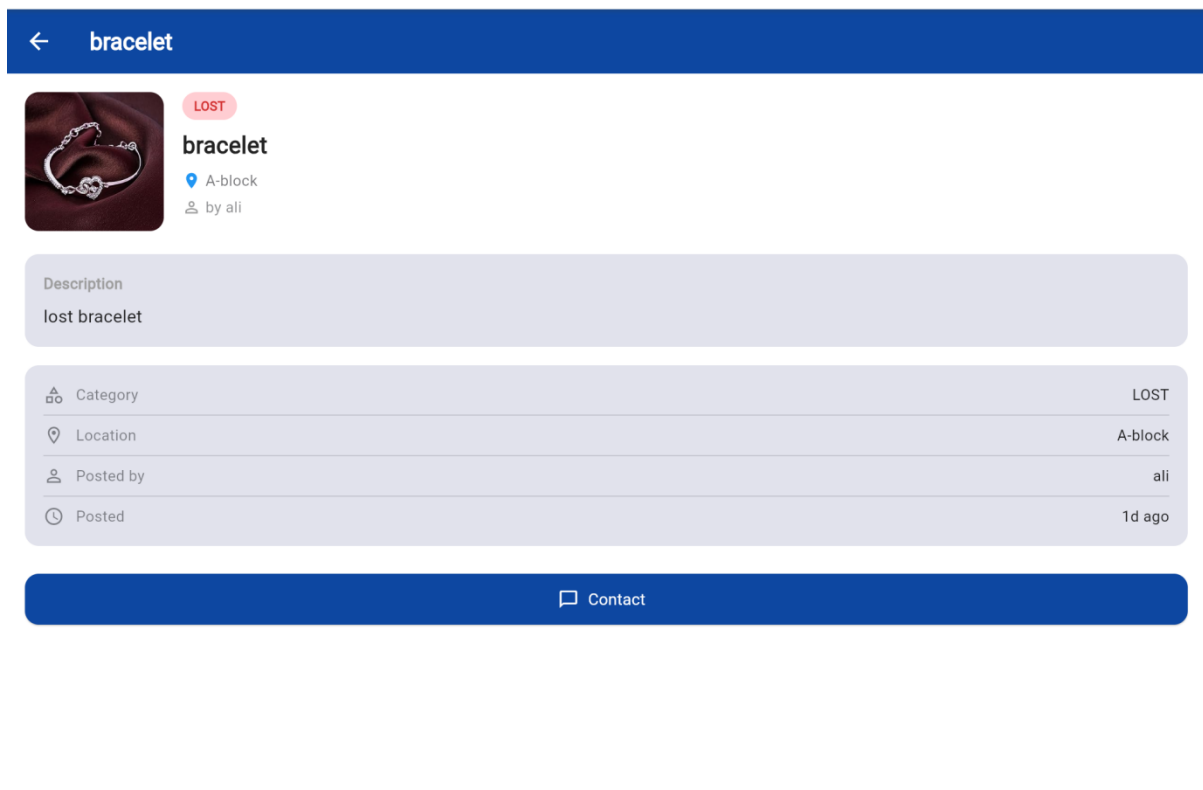


Figure 19: Item Detail Screen

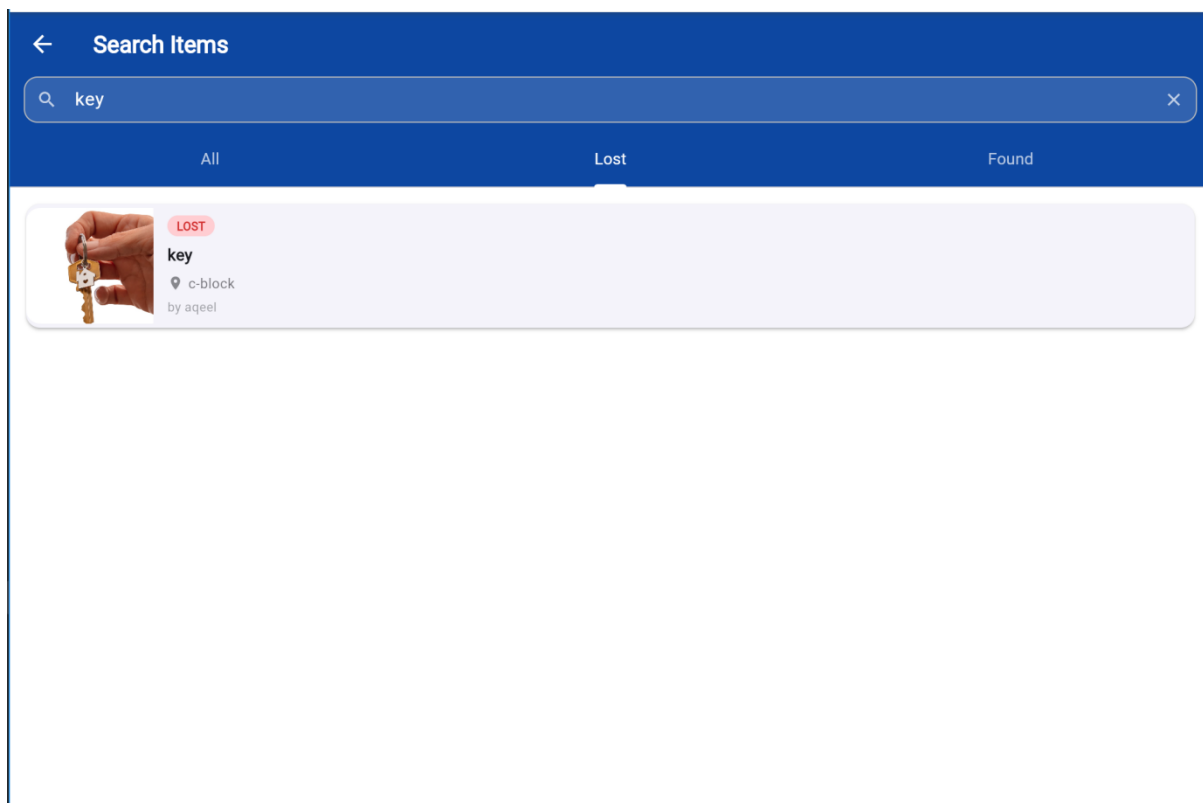


Figure 20: Search Screen

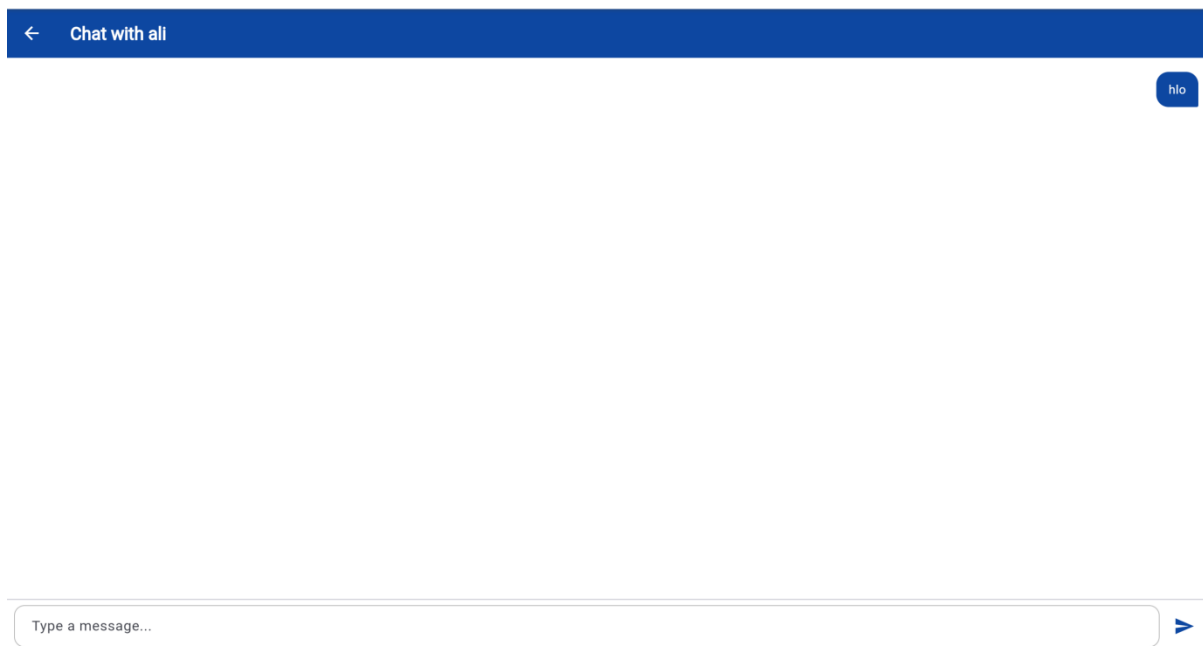


Figure 21: Chat Screen

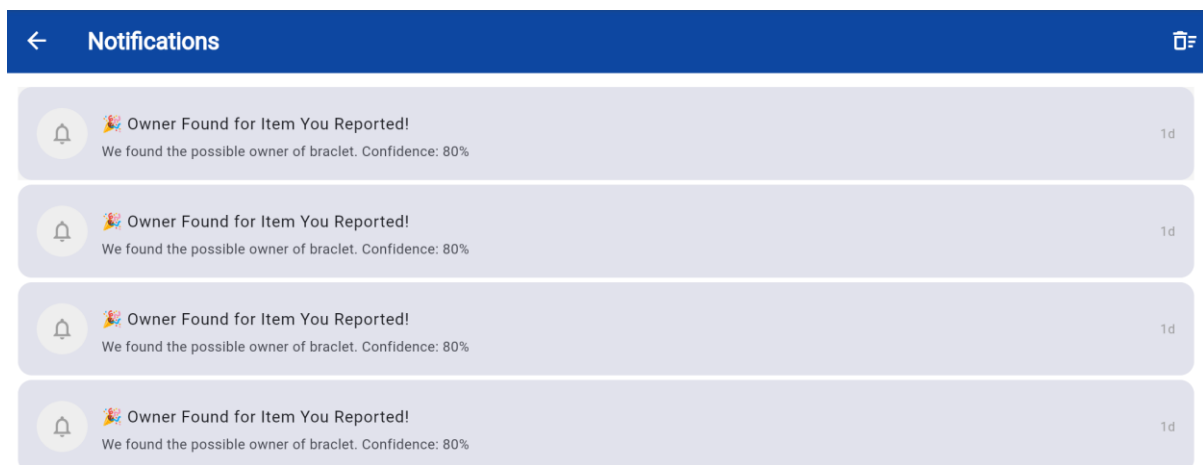


Figure 22: Notification Screen

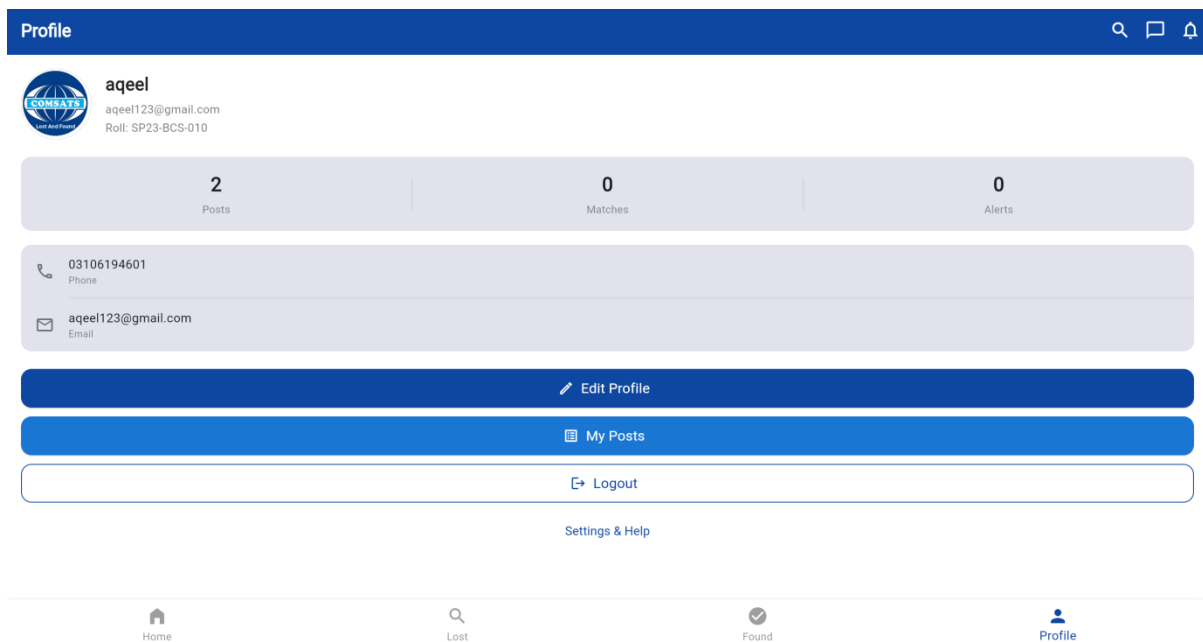


Figure 23: Profile Screen

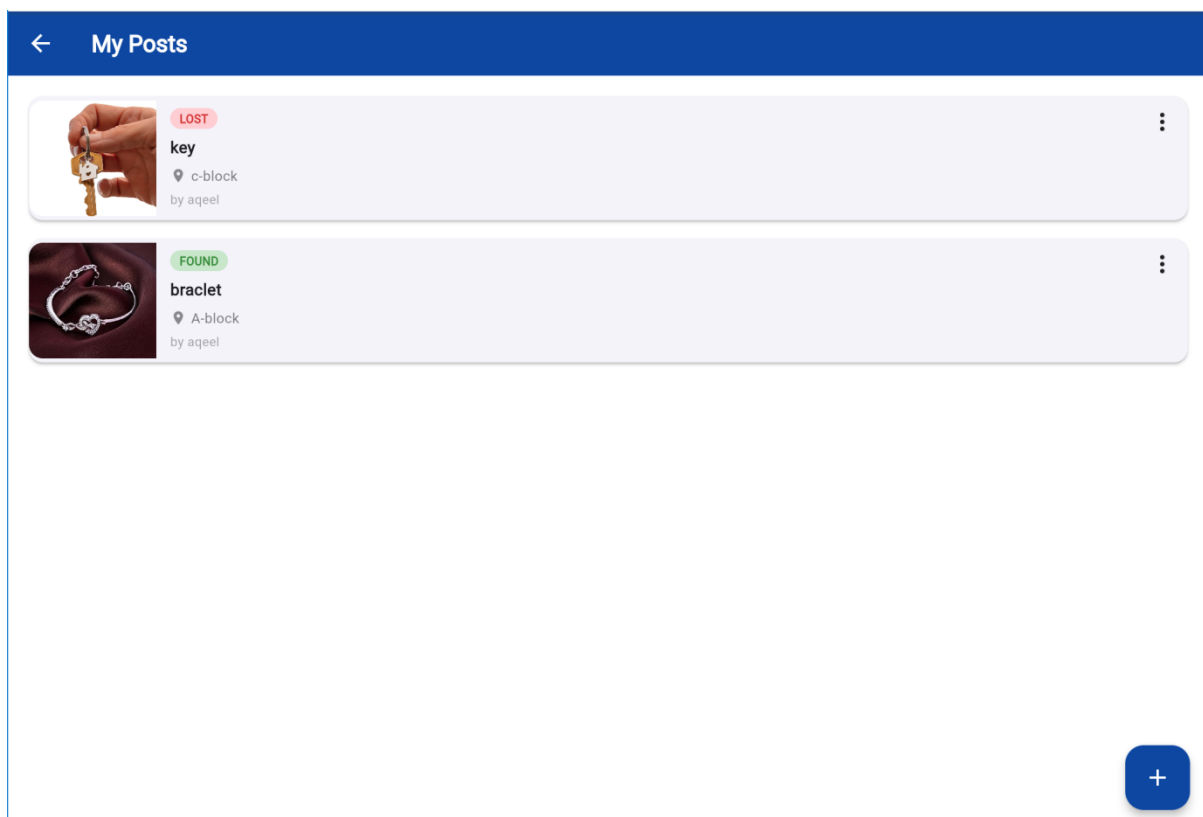


Figure 24: My Post Screen

← Edit Post

✓ Lost

Found



Item Photo

Add a clear photo to help with AI matching

Change Image

Title

key

Description

keys lost

Location

c-block

Update Post

Figure 25: Edit Post Screen

UniLost

localhost:60055/#/edit_profile

You are using an unsupported command-line flag: --no-sandbox. Stability and security will suffer.

← Edit Profile



Full Name

aqeel

Phone Number

03106194601

Roll Number

SP23-BCS-010

Save Changes

Search the web and Windows



Desktop 2:12 AM 5/13/2026

Figure 26: Edit Profile Screen

55

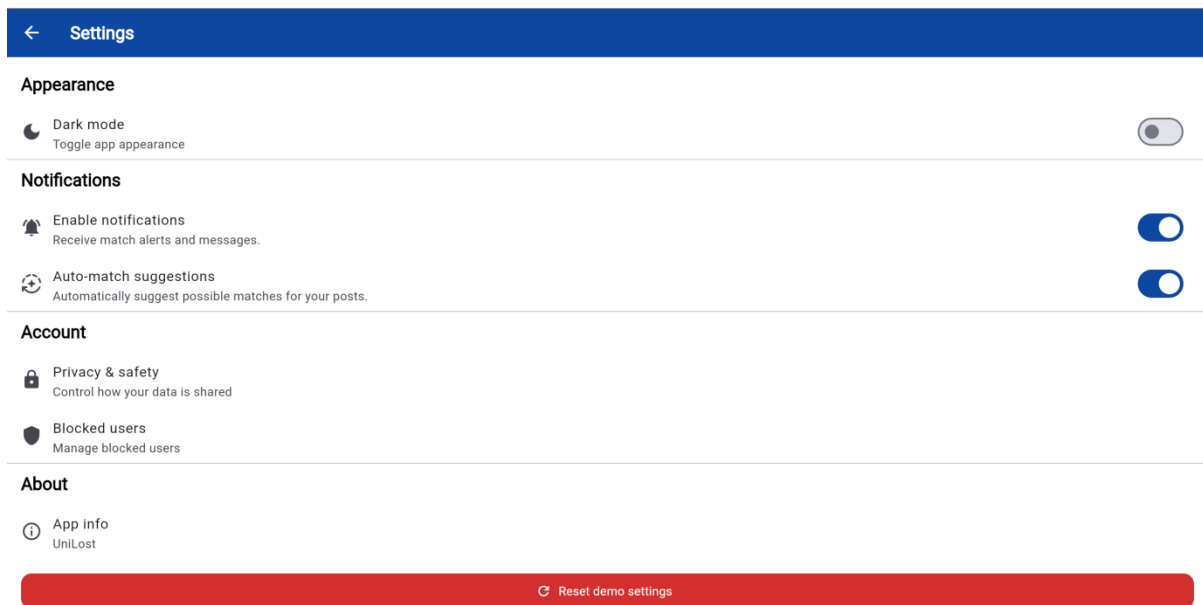


Figure 27: Settings Screen

12. Screen objects and actions

This section describes the major interface objects used in the UniLost application and the actions performed by users through these interface components. The interface objects include buttons, text fields, navigation controls, icons, and interactive elements used throughout the application.

The interface is designed to provide simple navigation, clear user interaction, and responsive feedback using Flutter widgets and GetX state management.

Table 13: Login and Signup Screen Objects

Object	Type	Action
Roll Number Field	Text Field	Accepts user roll number
Email Field	Text Field	Accepts user email
Password Field	Password Field	Accepts password input
Login Button	Button	Authenticates user
Signup Button	Button	Creates new account
Forgot Password	Text Button	Opens password reset flow

Table 14: : Home Screen Objects

Object	Type	Action
Search Icon	Icon Button	Opens search functionality
Notification Icon	Icon Button	Opens notifications screen

Chat Icon	Icon Button	Opens chats list
Bottom Navigation	Navigation Bar	Navigates between modules
Post Card	Card Widget	Opens item detail screen

Table 15: Create Post Screen Objects

Object	Type	Action
Title Field	Text Field	Accepts item title
Description Field	Text Area	Accepts item description
Location Field	Text Field	Accepts item location
Category Selector	Dropdown	Selects Lost or Found
Image Picker	Button	Opens camera/gallery
Submit Button	Button	Creates new post

Table 16: Chat Screen Objects

Object	Type	Action
Message Field	Text Field	Accepts message input
Send Button	Icon Button	Sends message
Messages List	List View	Displays chat messages
AI Message Widget	Text Widget	Displays AI-generated context

Table 17: Notifications Screen Objects

Object	Type	Action
Notification Card	Card Widget	Displays notification details
Read Status Icon	Icon	Shows read/unread state
Clear All Button	Button	Removes notifications
Notification List	List View	Displays all notifications

Table 18: Search Screen Objects

Object	Type	Action
Search Field	Text Field	Accepts search keywords
Search Results List	List View	Displays matching posts
Filter Tabs	Tabs	Filters Lost/Found items
Item Card	Card Widget	Opens item details

Table 19: My Posts and Edit Post Objects

Object	Type	Action
Edit Button	Icon Button	Opens edit post screen
Delete Button	Icon Button	Deletes post
Update Button	Button	Saves edited post

Image Picker	Button	Updates item image
--------------	--------	--------------------

Table 20: Profile and Settings Screen Objects

Object	Type	Action
Edit Profile Button	Button	Opens profile editing
Dark Mode Toggle	Switch	Enables/disables dark mode
Notifications Toggle	Switch	Enables/disables notifications
Logout Button	Button	Logs out current user
Settings Menu	Menu Option	Opens settings screen

13. Conclusion

The Software Design Description of the UniLost system presents the complete technical design of an AI-powered Lost and Found platform developed exclusively for COMSATS University Islamabad, Sahiwal Campus students.

The document covers the system architecture, process flows, design models, database structure, algorithms, interface design, and requirements traceability. The system is designed using Flutter, Firebase, Clouinary, and Google Gemini AI to provide a scalable, intelligent, and secure lost and found platform for the university community [1][2][7][8].

The design is structured around three core pillars. The first is intelligent matching where Gemini AI performs multimodal comparison of lost and found posts using both image visual analysis and text semantic comparison to automatically identify potential matches. The second is trust and verification where a multi-layer ownership verification system, QR code handover, strike system, and student trust scores ensure the integrity and security of every item recovery. The third is campus oversight where the admin dashboard provides university administrators with complete visibility and control over posts, users, matches, reports, and campus-wide analytics.

The MVVM architectural pattern with GetX state management and Repository Pattern ensures the system remains maintainable, scalable, and modular for future enhancements [9]. The Agile development methodology allowed continuous refinement of features throughout the development cycle based on supervisor feedback [14].

Currently the major frontend and backend functionalities of the system have been implemented. Remaining work includes completion of the match confirmation screen, ownership verification screen, meetup scheduling screen, QR handover screen, chatbot interface, and admin web dashboard, along with final system testing and deployment.

The SDD ensures that all functional requirements defined in the SRS are fully translated into a structured technical design ready for implementation and future development.

14. References

- [1] Flutter Documentation — <https://flutter.dev>
- [2] Firebase Documentation — <https://firebase.google.com/docs>
- [3] Cloud Firestore Documentation — <https://firebase.google.com/docs/firestore>
- [4] Firebase Authentication Documentation — <https://firebase.google.com/docs/auth>
- [5] Firebase Cloud Functions Documentation — <https://firebase.google.com/docs/functions>
- [6] Firebase Cloud Messaging Documentation — <https://firebase.google.com/docs/cloud-messaging>
- [7] Google Gemini API Documentation — <https://ai.google.dev>
- [8] Cloudinary Documentation — <https://cloudinary.com/documentation>
- [9] GetX State Management Documentation — <https://pub.dev/packages/get>
- [10] Material Design 3 Guidelines — <https://m3.material.io>
- [11] Dart Programming Language Documentation — <https://dart.dev>
- [12] Geolocator Flutter Package — <https://pub.dev/packages/geolocator>
- [13] QR Code Scanner Flutter Package — https://pub.dev/packages/qr_code_scanner
- [14] Agile Software Development — Sommerville I. Software Engineering 9th Edition
- [15] UML Modeling Guidelines — <http://agilemodeling.com/artifacts>